Observer–Actor Dynamic: Technical Report

Abstract

This report presents controlled experiments for the Observer–Actor learning framework. Each experiment separates one learned part of the system and records both aggregate reward and the specific quantity that part is supposed to learn.

The first two experiments study reward-weighted policy learning in a single fixed state and in multiple fully observed states. The one-to-one imperfect-state experiment starts the imperfect-state track: an Actor observes the state and sends a scalar compressed message, while an Observer acts from that message. The imperfect-reward experiments study a single Observer with a limited budget for following many Actors, followed by the group-level version with many Observers and imperfect reward functions. The imperfect-state experiments then scale the state-information problem: one Observer selects among several Actors that send messages, and then several Observers each select an Actor's message and act. The last experiment combines the two information problems. Each agent sends state information, selects a peer message, and acts, while reward Observers decide how strongly to follow the acting agents and provide the feedback used for learning.

Contents

1 Overview **3**

2 Method **3**

2.1 Imperfect-State Implementation 4

2.1.1 State, message, action, and reward 4

2.1.2 Policy for a scalar message 5

2.1.3 Selection, exploration, and active updates 5

2.1.4 Convergence records 6

3 Single-State Action Learning **7**

4 State-Dependent Action Learning **8**

5 Imperfect State with One Actor and One Observer **10**

5.1 Learning the State Message with a Fixed Policy 11

5.2 Learning the Policy with a Fixed Correct Message . . . 11
5.3 Learning Compression and Policy Together . . . 11
5.4 Diagonal Scale Ladder with Ceiling Rulers . . . 12

6 Imperfect Reward: One Observer and Many Actors 13
6.1 Learnable Following Rates with Fixed Actor Policies . . . 14
6.2 Learnable Following Rates and Actor Policies . . . 16

7 Group-Level Interactions: Imperfect Reward 17
7.1 Multiple Observers and Actors with Learnable Following Rates . . . 17
7.2 Multiple Observers and Actors with Joint Learning . . . 18

8 Imperfect State with Multiple Actors and One Observer 19
8.1 Learning Compression Only . . . 19
8.2 Learning Selection Only . . . 20
8.3 Learning Policy Only . . . 20
8.4 Learning Compression and Selection . . . 20
8.5 Learning Selection and Policy . . . 20
8.6 Learning Compression and Policy . . . 20
8.7 Learning Compression, Selection, and Policy . . . 20

9 Imperfect State with Multiple Actors and Multiple Observers 22
9.1 Learning Compression Only . . . 22
9.2 Learning Selection Only . . . 22
9.3 Learning Policy Only . . . 22
9.4 Learning Compression and Selection . . . 22
9.5 Learning Selection and Policy . . . 22
9.6 Learning Compression and Policy . . . 22
9.7 Learning Compression, Selection, and Policy . . . 23

10 Combined Imperfect State and Imperfect Reward 25
10.1 Color-coded status . . . 25
10.2 Setup, architecture, and initialization . . . 27
10.3 Sixteen-rung ladder at $K = A = 3$ . . . 28

10.4 State and population scaling 29

11 Discussion **30**

A Additional Experiments **30**

1 Overview

The paper defines Actor and Observer as interaction roles. The Actor produces the action being evaluated, and the Observer evaluates that action and returns feedback. In the imperfect-reward setting, the Actor takes an environmental action and the Observer reports reward. In the imperfect-state setting, the Actor takes an informational action by sending a compressed state message; the Observer uses that message to take the environmental action.

There are two main information problems in the report. In the imperfect-reward cases, each Actor can see the state and choose actions, but reward feedback comes through Observer following. The learned quantities are the following rates μ, estimated Actor utilities $\hat{U}$, and in the joint cases the Actor policy. In the imperfect-state cases, the direction is different. An Actor sees state information and sends a compressed message $m = f_\phi(s)$. The Observer receives the message, chooses an environmental action, and sends reward feedback. When several Actors are available, the Observer also learns which Actor to listen to. Those information-source selection probabilities are denoted by λ. The report uses the paper roles Actor and Observer; it does not use informal labels such as "active compressor" or "auto compressor."

The experiments follow a ladder: policy learning first, then scalar compression, then selection among message-sending Actors, then several Observers. The combined experiment puts imperfect reward reporting on top of the group imperfect-state interaction, so each quantity can be checked alone before they are opened together.

Reward alone is not enough. We also track message values, compression weights, following rates μ, selection probabilities λ, and per-state action probabilities. A run counts as successful only when reward and the relevant internal quantity both move the right way.

In the combined setting the four decision quantities are compression ϕ, peer selection w, policy θ, and reward following μ. We learn them one at a time, then in pairs and triples, then all four. Section 10 marks each claim black, green, or red: black is trustworthy on the stated task, green was broken and then fixed, and red is an open pressure point.

2 Method

This section fixes the common notation used in the experiments. An environmental policy maps its input to action logits, and the softmax of those logits gives the action probabilities. In the imperfect-reward setting this is the Actor policy. In the imperfect-state setting this is the Observer policy. Some controlled experiments fix this policy so that only compression, reward-following rates μ, or information-source selection probabilities λ are learned. When the policy is learned, it is updated from sampled rewards unless the text explicitly marks the run as a controlled comparison.

For the action-learning cases, the training loss is

$$\mathcal{L}_t = -\hat{r}_t \log \pi_\theta(a_t \mid x_t),$$

where $\hat{r}_t$ is the Observer-reported reward, x_t is the Actor input, and a_t is the sampled action. Minimizing this loss performs stochastic gradient ascent on expected reward:

$$\theta_{t+1} \leftarrow \theta_t + \alpha r_t \nabla_\theta \log \pi_\theta(a_t \mid x_t).$$

The policy input differs by case:

$$x_t = \begin{cases} s_{\text{static}}, & \text{single-state action learning,} \\ s_t, & \text{state-dependent action learning,} \\ m_t = f_\phi(s_t), & \text{one-to-one scalar-compression experiments,} \\ m_t = f_{\phi_{k_t}}(s_t), & \text{multi-Actor imperfect-state experiments.} \end{cases}$$

2.1 Imperfect-State Implementation

All imperfect-state numbers in the following sections were generated after the state, action, and Actor-selection random streams were separated. Each stream receives a deterministic seed derived from the run seed and its role. This matters because an earlier implementation seeded state and action sampling with the same random stream. The present experiments do not use those earlier results.

2.1.1 State, message, action, and reward

There are K states. State k is represented by a cumulative binary vector with k leading ones. For $K = 5$, the states are

k	s_k	$\sum_q s_{k,q}$
0	$[0, 0, 0, 0, 0]$	0
1	$[1, 0, 0, 0, 0]$	1
2	$[1, 1, 0, 0, 0]$	2
3	$[1, 1, 1, 0, 0]$	3
4	$[1, 1, 1, 1, 0]$	4.

An Actor i observes either no coordinates, a partial set of coordinates, or the full state. It sends a scalar message

$$m_i(s) = \phi_i^\top s + b_i.$$

The full Actor can in principle represent the coordinate sum, but the experiments learn ϕ_i from reward feedback; no loss directly matches its message to the state index.

The target action is $a^\star(k) = k$ when $A = K$. When fewer actions are used, consecutive states share an action:

$$a^\star(k) = \min\{A - 1, \lfloor kA/K \rfloor\}.$$

For example, with $K = 64$ and $A = 16$, states 0 through 3 have target action 0, states 4 through 7 have target action 1, and so on. A correct action receives reward 1, and any other action receives -0.1.

2.1.2 Policy for a scalar message

The Observer policy uses an adaptive radial-basis-function policy. A radial basis function is a smooth bell-shaped feature of a scalar. The policy first normalizes the message and applies a learned source-specific affine map,

$$z_i(m) = \rho_i \frac{m - \bar{m}}{\sqrt{v_m + 10^{-8}}} + \delta_i,$$

then computes B features,

$$h_{i,r}(m) = \exp\left[-\frac{1}{2}\left(\frac{z_i(m) - c_{i,r}}{\sigma_{i,r}}\right)^2\right], \qquad r = 1, \ldots, B.$$

Here $c_{i,r}$ is a learned feature center and $\sigma_{i,r} > 0$ is its learned width. The action logits and probabilities are

$$\ell_i(m) = W_i h_i(m) + d_i, \qquad \pi_{\theta_i}(a \mid m) = \mathrm{softmax}(\ell_i(m))_a.$$

Thus the learned policy consists of the feature centers, widths, affine parameters, and action-layer weights. If several Actors are available, each source has its own such policy parameters. The selected source is known to the Observer, so this conditions on information already available at the action step.

For a small numerical example, suppose a normalized scalar is $z = 1.1$, a feature center is $c = 1.0$, and its width is $\sigma = 0.2$. That feature has value $\exp[-(0.1)^2/(2 \cdot 0.2^2)] \simeq 0.882$. The learned action layer gives each action a different weight on this feature and on all other features. The softmax then turns those action scores into probabilities. The message stays continuous throughout this calculation, so the policy-gradient update can also change the compression weights through $m_i(s)$.

The number of features is $B = \max\{64, 2K\}$. Centers are initialized independently from a standard normal distribution. Widths start at 0.2 in the multi-Actor imperfect-state experiments, the affine scale starts at 1, the affine shift starts at 0, and the action layer uses its standard random PyTorch initialization. The initial feature widths are bounded during training by 1.5 times the mean adjacent normalized-message gap. This keeps a feature from becoming so wide that neighboring scalar message regions are indistinguishable. No state-action correspondence is inserted into this initialization.

Fixed-policy controls use a separate, non-learned reference policy. It gives the largest score to the action center closest to the scalar message. This provides a fixed action rule when the experiment is intended to isolate compression or selection rather than policy learning.

2.1.3 Selection, exploration, and active updates

With C Actors, an Observer selects source i from $\lambda(i)$. Training samples from a mixture with a uniform floor,

$$\widetilde{\lambda}(i) = (1 - \epsilon_w)\lambda(i) + \frac{\epsilon_w}{C}.$$

Likewise, actions are sampled from

$$\widetilde{\pi}(a \mid m) = (1 - \epsilon_a)\pi_\theta(a \mid m) + \frac{\epsilon_a}{A}.$$

Whenever an object is learned, the corresponding sampling distribution uses a uniform floor of 0.20; a floor is zero when that object is fixed. The same mixed distributions are used for sampling and in the REINFORCE log probability. This avoids treating exploration draws as if they had come from the unmodified learned policy.

The active objects are updated only in the subcase that names them. Policy and compression use Adam; selection weights use stochastic gradient descent. The common base learning rate is 0.001, unless a result table says otherwise. Compression and policy rates follow a cosine schedule to zero over the first 80% of a run. The remaining 20% is an observation window, not additional learning. Replay, supervised action labels, critic baselines, entropy terms, and message-separation losses are all disabled.

2.1.4 Convergence records

Each run saves its random-stream seeds, initial and final compression parameters, emitted messages, adaptive-policy parameters, selection probabilities, action probabilities, and optimizer state. A run is marked stable only when the last five recorded checkpoints have reward range at most 0.01, message movement at most 0.01, correct-action-probability movement at most 0.01, selection-probability movement at most 0.01, and relative parameter movement at most 0.01 for every active object. Because the cosine rates are zero in this final window, this is a direct check that the learned mappings and parameters have settled.

The reported internal measures are the lowest probability assigned to a correct action over all states, the number of state-action regions below 0.80, and $P(\text{full})$, the probability assigned to the full Actor. Expected reward is evaluated without the exploration mixture. It therefore measures the learned policy, while the floors are used only to obtain training samples.

Where a scale comparison needs an oracle ceiling, two rulers are used at $p_{\text{full}} = 1$. $Y_{\text{fixed sum}}$ is the reward of the correct coordinate-sum message under the non-learned sum decoder. Y_{RBF} is the reward of the adaptive RBF policy with the message held at that same correct sum. A learned run Z that matches Y_{RBF} but falls below $Y_{\text{fixed sum}}$ is limited by the learned policy head, not by missing state information. Paper-only contrast rows replace the adaptive RBF head and Adam updates with the fixed sum-shaped softmax head and SGD, and they turn off the action exploration floor.

The imperfect-reward experiments use a separate setup: the state is fully observed, and the learned objects are budgeted following rates μ, utility estimates $\hat{U}$, and in the joint case Actor policies.

For the first two action-learning experiments, the default learning-rate grid is

$$\{0.001,\ 0.01,\ 0.05,\ 0.1\},$$

with 5 random seeds and 5000 training steps. Later experiments state their own learning rates, random seeds, and training steps. The main reward metric is expected reward under the current policy. The figures also report the learned quantity required by the case: action probabilities in the action-learning experiments, message behavior in the one-to-one imperfect-state experiment, information-source selection probabilities and correct-action probability in the imperfect-state experiments, and reward-following rates μ in the imperfect-reward experiments.

The tables use consistent metric definitions. Learning rate is the optimizer step size. Expected reward is the reward the current stochastic policy would receive on average before sampling an

action. In one state this is $\sum_a \pi(a \mid x) r(a)$; across states it is also averaged over the state distribution. End expected reward is this quantity at the last reported training step, averaged over seeds. End sampled reward averages the last 100 rewards actually sampled during training. High-reward action probability is the total probability assigned to the maximum-reward action after averaging over the relevant states and selected Actors. Success rate is the fraction of seeds reaching at least 95% of the reward ceiling. Plots show trajectories; tables summarize end-of-run behavior.

For example, suppose one state has rewards $r(0) = 0$, $r(1) = 1$, and $r(2) = 0.2$, and the current policy assigns probabilities $(0.05, 0.90, 0.05)$. The expected reward is $0.05 \cdot 0 + 0.90 \cdot 1 + 0.05 \cdot 0.2 = 0.91$, even though the next sampled reward will be one of 0, 1, or 0.2. In a three-seed run, an end expected reward of 0.96 means the three end expected rewards average to 0.96. If the reward ceiling is 1.0, a seed is counted as successful once it reaches at least 0.95. If high-reward action probability is 0.90, the current policy assigns 90% probability to the best action after receiving the state input or selected Actor message.

3 Single-State Action Learning

This case tests the basic policy-gradient update. There is one fixed state s_{static}, three actions, and the reward table is

$$r(0) = 0.0, \qquad r(1) = 1.0, \qquad r(2) = 0.2.$$

The only learned object is the Actor's action distribution in this one state. The objective is

$$J(\theta) = \mathbb{E}_{a \sim \pi_\theta(\cdot \mid s_{\text{static}})}[r(a)] = \sum_a \pi_\theta(a \mid s_{\text{static}}) r(a).$$

The optimal behavior is to put almost all probability on action 1.



Figure 1: Single-state reward learning curves. Expected reward gives the policy-level performance, while average sampled reward shows the recent reward actually experienced during training. Curves show the mean over 5 seeds for each learning rate.

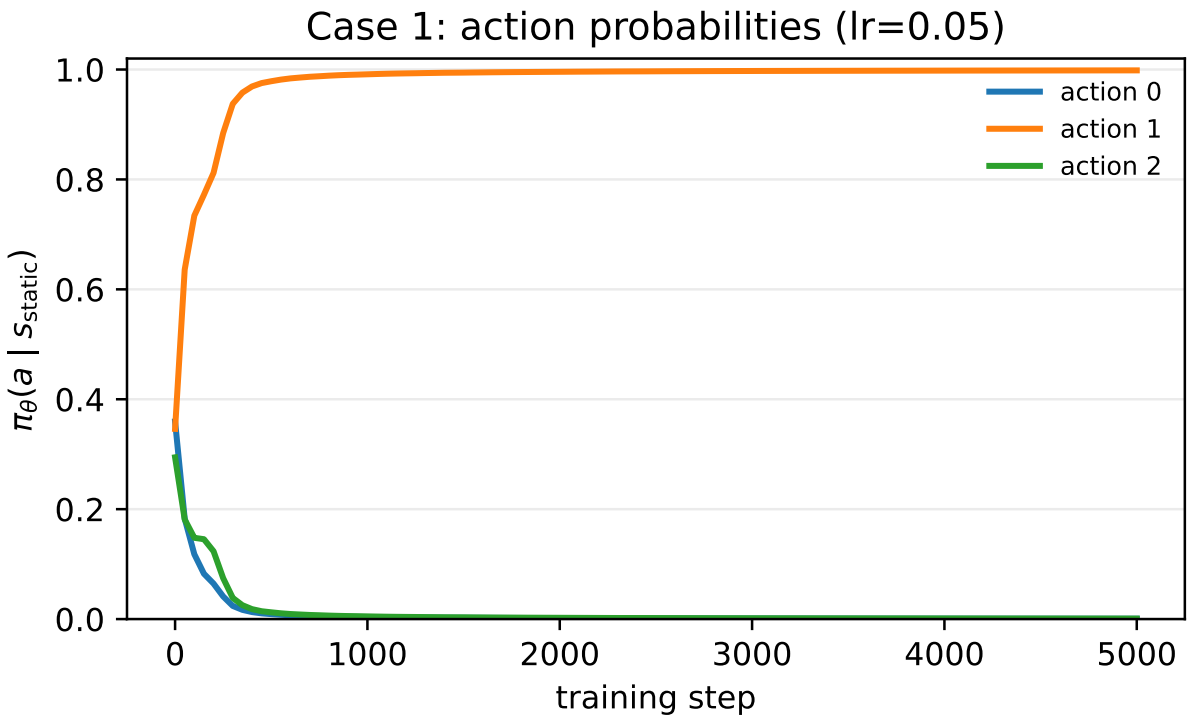


Figure 2: Single-state action probabilities. The probability of the reward-maximizing action rises toward one, while the lower-reward actions lose probability. Curves show the mean over 5 seeds at learning rate 0.05.

Learning rate	End expected	End sampled	Success rate
0.001	0.772	0.795	0.00
0.01	0.992	0.989	1.00
0.05	0.999	0.996	1.00
0.1	0.999	0.998	1.00

Table 1: Single-state action-learning end-of-run summary over 5 seeds.

This shows the intended update directly: the learned object is the action distribution, and the probability of the best action increases.

4 State-Dependent Action Learning

The state-dependent action-learning experiment tests whether the same update can learn different actions in different fully observed states. There are three states, sampled uniformly, and three

actions. States are represented as state vectors. The reward structure is:

	$a=0$	$a=1$	$a=2$
$s=0$	1.0	0.0	0.2
$s=1$	0.1	1.0	0.0
$s=2$	0.0	0.2	1.0

The learned object is now a conditional policy: for each state, the Actor should put high probability on that state's best action.

The objective is

$$\begin{aligned} J(\theta) &= \mathbb{E}_{s\sim D}\left[\mathbb{E}_{a\sim\pi_\theta(\cdot|s)}[r(s,a)]\right] \\ &= \sum_s D(s) \sum_a \pi_\theta(a \mid s) r(s,a). \end{aligned}$$

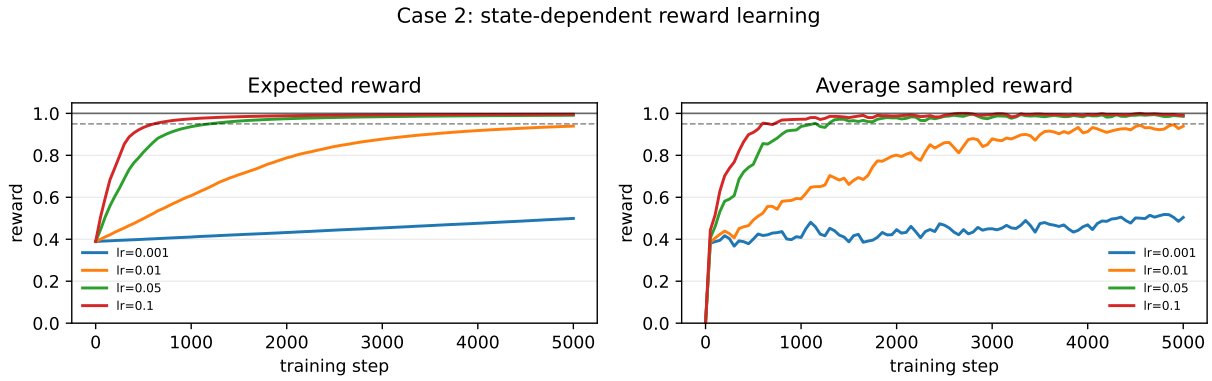


Figure 3: State-dependent reward learning curves. Expected reward gives the policy-level performance across the uniformly sampled states, while average sampled reward shows the recent reward actually experienced during training. Curves show the mean over 5 seeds for each learning rate.

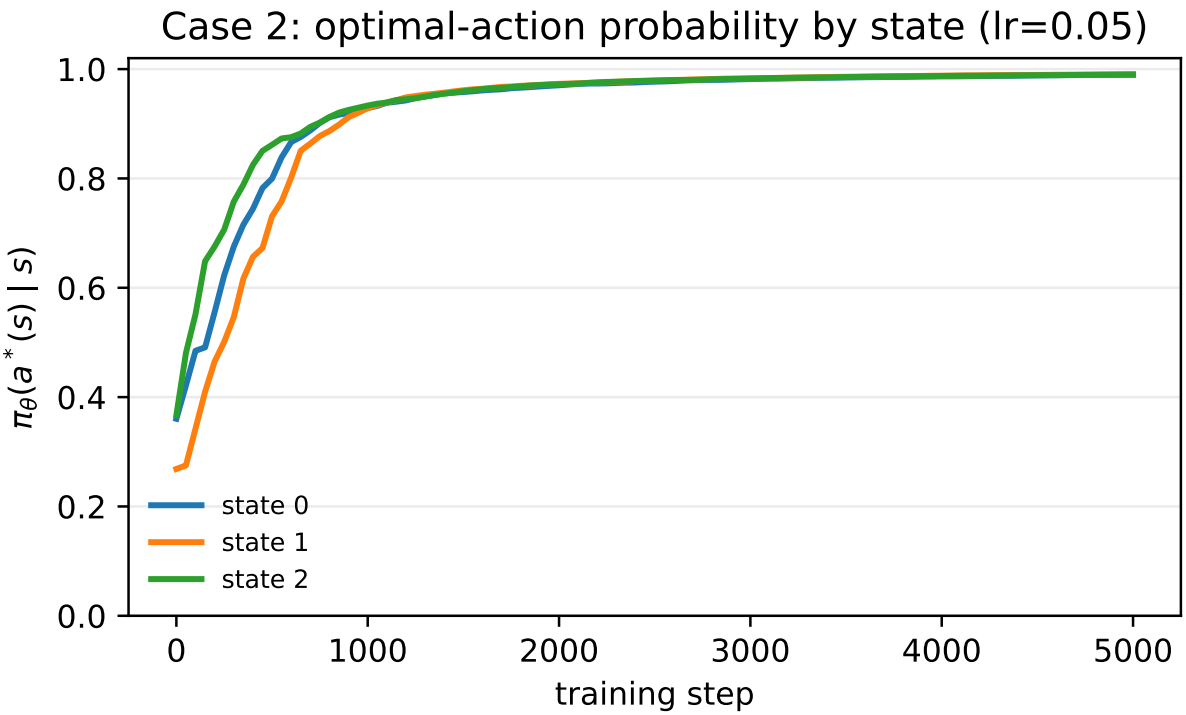


Figure 4: State-conditioned action probabilities. Each curve shows the probability assigned to the correct action for one state. The model learns the three state-action relationships while average reward improves. Curves show the mean over 5 seeds at learning rate 0.05.

Learning rate	End expected	End sampled	Success rate
0.001	0.499	0.504	0.00
0.01	0.939	0.939	0.00
0.05	0.991	0.986	1.00
0.1	0.996	0.991	1.00

Table 2: State-dependent action-learning end-of-run summary over 5 seeds.

The per-state probability plot shows the internal behavior that the scalar reward curve hides: each conditional action distribution learns its own best action.

5 Imperfect State with One Actor and One Observer

This section has one full-state Actor and one Observer. There is no selection weight because the Observer has only one available message source. The three experiments add one learned object at a time: compression ϕ, policy θ, and then both together. All results use five independent seeds. The message is scalar, the action count equals the state count, and the reward is 1 for the target action and -0.1 otherwise.

5.1 Learning the State Message with a Fixed Policy

Only the compression weights ϕ are learned. The Observer uses the fixed reference policy described in the Method section. Its finite sharpness means that a correct message has correct-action probability near 0.965, not exactly 1. The policy is fixed so that any improvement comes from the learned scalar message.

5.2 Learning the Policy with a Fixed Correct Message

Only the adaptive radial-basis-function policy θ is learned. The message is held at the correct coordinate sum $m = k$. This establishes that the policy can learn the action mapping when the message does not move.

5.3 Learning Compression and Policy Together

Both ϕ and θ are learned from sampled reward. Their learning rates are equal in each row. The initial compression weights are positive random values, $\phi_q \sim \text{Uniform}(0, 1/\sqrt{K})$, with $b \sim \text{Uniform}(-1/\sqrt{K}, 1/\sqrt{K})$. This gives an ordered random scalar code for the cumulative states, but does not provide the target messages or target actions.

Learned object	$K = A$	Steps	Learning rate	ϵ_a	Reward	Lowest correct	Seeds with no region below 0.80
Compression ϕ	3	100k	$\alpha_\phi = 0.010$	0.00	0.987	0.965	5/5
Policy θ	3	100k	$\alpha_\theta = 0.003$	0.10	1.000	1.000	5/5
ϕ, θ	3	100k	$\alpha_\phi = \alpha_\theta = 0.003$	0.10	1.000	1.000	5/5
Compression ϕ	5	150k	$\alpha_\phi = 0.005$	0.00	0.977	0.965	5/5
Policy θ	5	100k	$\alpha_\theta = 0.003$	0.10	1.000	1.000	5/5
ϕ, θ	5	100k	$\alpha_\phi = \alpha_\theta = 0.003$	0.10	0.996	0.993	5/5
Compression ϕ	8	300k	$\alpha_\phi = 0.001$	0.20	0.939	0.839	5/5
Policy θ	8	200k	$\alpha_\theta = 0.001$	0.20	1.000	0.998	5/5
ϕ, θ	8	300k	$\alpha_\phi = \alpha_\theta = 0.001$	0.20	0.976	0.944	5/5

Table 3: One-to-one scalar-message experiments with independent random streams. Reward and lowest correct-action probability are means over five seeds. The reported runs meet the final-window stability criterion in the Method section.

Figure file `rng_clean_section5_one_to_one_summary` is not in the repository yet.

Figure 5: One-to-one learning through eight states and actions. The panels show expected reward and the lowest correct-action probability for each learned-object setting. The final movement panels use the last checkpoints after the cosine learning rate reaches zero.

At $K = 8$, reducing the common learning rate to 0.001, retaining the uniform action floor, and increasing the training horizon produce stable solutions in all five seeds. The learned joint messages need not equal the literal coordinate sum. What the result establishes is that each state message leads to its correct action with probability above 0.90, while the recorded compression and policy parameters are stable in the final window.

5.4 Diagonal Scale Ladder with Ceiling Rulers

The previous table stops at $K = A = 8$. A separate diagonal ladder extends the same one-to-one setting through $K = A \in \{4, 8, 16, 32, 64, 128\}$ with three seeds per cell. Step budgets grow with K: 300k for $K \leq 8$, 400k for $K = 16$, 800k for $K = 32$, 1.6M for $K = 64$, and 3.2M for $K = 128$. The engineering recipe matches the Method section (adaptive RBF, Adam, action floor 0.20, positive-random compression). A paper-only contrast uses the fixed sum-shaped softmax head with SGD and no action floor.

Two oracle ceilings at $p_{\text{full}} = 1$ separate message failure from policy-head failure. $Y_{\text{fixed sum}}$ is the expected reward of the correct coordinate-sum message under the non-learned sum decoder. Its value stays near 0.96–0.97 on the diagonal. Y_{RBF} is the mean reward of the adaptive RBF policy when the message is held at the correct sum (the θ-only cell). Joint ϕ, θ learning should be read against both rulers.

Learned object	$K = A$	Reward Z	$Y_{\text{fixed sum}}$	Z/Y_{fs}	Y_{RBF}	Z/Y_{RBF}
Compression ϕ	4	0.981	0.971	1.01	1.000	0.98
Policy θ	4	1.000	0.971	1.03	1.000	1.00
ϕ, θ	4	1.000	0.971	1.03	1.000	1.00
Compression ϕ	8	0.971	0.966	1.01	1.000	0.97
Policy θ	8	1.000	0.966	1.04	1.000	1.00
ϕ, θ	8	1.000	0.966	1.04	1.000	1.00
Compression ϕ	16	0.966	0.964	1.00	1.000	0.97
Policy θ	16	1.000	0.964	1.04	1.000	1.00
ϕ, θ	16	0.977	0.964	1.01	1.000	0.98
Compression ϕ	32	0.919	0.962	0.96	0.632	1.46
Policy θ	32	0.632	0.962	0.66	0.632	1.00
ϕ, θ	32	0.662	0.962	0.69	0.632	1.05
Compression ϕ	64	0.962	0.962	1.00	0.472	2.04
Policy θ	64	0.472	0.962	0.49	0.472	1.00
ϕ, θ	64	0.470	0.962	0.49	0.472	1.00
Compression ϕ	128	0.962	0.961	1.00	0.387	2.49
Policy θ	128	0.387	0.961	0.40	0.387	1.00
ϕ, θ	128	0.378	0.961	0.39	0.387	0.98

Table 4: One-to-one diagonal scale ladder under the adaptive-RBF engineering recipe (three seeds). Through $K = 16$, joint learning stays near both ceilings. From $K = 32$ onward, ϕ-only remains near the fixed-sum decoder, while θ-only and joint ϕ, θ fall together and track Y_{RBF}. The diagonal failure is therefore a policy-head limit under $A = K$, not a failure to learn useful messages.

Learned object	$K = A$	Adaptive-RBF Z	Paper-only Z	$Y_{\text{fixed sum}}$	Y_{RBF}
ϕ, θ	4	1.000	0.995	0.971	1.000
ϕ, θ	8	1.000	0.611	0.966	1.000
ϕ, θ	16	0.977	0.239	0.964	1.000
ϕ, θ	32	0.662	0.075	0.962	0.632
ϕ, θ	64	0.470	−0.002	0.962	0.472
Policy θ	16	1.000	0.123	0.964	1.000
Policy θ	32	0.632	0.002	0.962	0.632
Compression ϕ	64	0.962	0.879	0.962	0.472
Compression ϕ	128	0.962	0.337	0.961	0.387

Table 5: Selected diagonal contrasts between the adaptive-RBF recipe and the paper-only softmax/SGD recipe (three seeds). Paper-only joint learning collapses much earlier than the adaptive-RBF recipe. Compression-only remains comparatively strong under both recipes until the largest sizes.

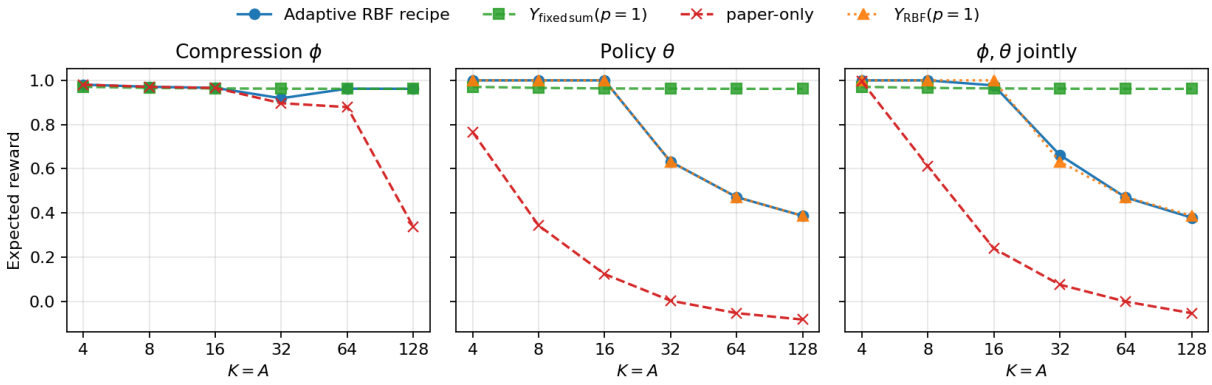


Figure 6: One-to-one diagonal scale ladder with $A = K$. Each panel is one learned object. Solid markers are the adaptive-RBF recipe; crosses are paper-only. The dashed green curve is the fixed-sum decoder ceiling; the dotted orange curve is the adaptive-RBF oracle-message ceiling Y_{RBF}.

These diagonal numbers must not be confused with the one-to-many tables, where $A < K$ groups neighboring states onto shared actions. In particular, $K = 64, A = 16$ succeeds under the same engineering recipe, while $K = A = 64$ does not. The gap is the harder one-action-per-state mapping, not a contradiction between sections.

6 Imperfect Reward: One Observer and Many Actors

This section corresponds to the paper's one-Observer imperfect-reward setting. Every Actor observes the full state. After an Actor acts, the Observer sees the reward for that action and reports it with probability determined by the following rate. A single Observer watches N Actors and has a limited budget for deciding which Actors to follow.

A useful way to read this section is as one instructor watching several students. The students are the Actors: they produce answers. The instructor is the Observer: it estimates which students tend to produce higher-reward answers and spends more of its limited following budget on those students. This picture is only an intuition, but it matches the roles in the update: $\hat{U}_j$ estimates Actor j's average quality, and μ_j controls how often the Observer gives that Actor reward feedback.

The new learnable object is the budgeted following-rate vector

$$\mu = (\mu_1, \ldots, \mu_N), \qquad \mu_j \geq 0, \qquad \sum_{j=1}^{N} \mu_j \leq B,$$

where μ_j is how strongly the Observer follows Actor j. The following rate sets the probability that the Observer actually reports the reward it saw,

$$q(\mu_j) = 1 - e^{-c\,\mu_j}, \qquad c > 0,$$

so larger attention means more frequent feedback. The Observer also keeps a running utility estimate $\hat{U}_j$ of each Actor's average reward. The tracked variables for this case are $\mu_j(t)$ and $\hat{U}_j(t)$: the main experiment is reward improvement together with Observer reallocation toward the genuinely higher-utility Actors.

Setup conventions. The controlled tests use a single state (so each fixed Actor's utility is known analytically), three actions, scalar state-ID inputs mapped through a learnable embedding, and a hard-coded reward table. One Actor $j_t \sim p$ acts per step with p uniform; the Observer sees only the realized reward of the acting Actor. In the fixed-policy experiment, the Actor policies are fixed and only the Observer's μ and $\hat{U}$ are learned. In the joint-learning experiment, each Actor additionally learns its softmax policy.

Updates. When Actor j acts, the utility estimate uses a fast stochastic-approximation step

$$\hat{U}_{j,t+1} = \hat{U}_{j,t} + \alpha_U\big(r_t - \hat{U}_{j,t}\big),$$

while the following rates take a slower projected-gradient-ascent step on $\sum_j q(\mu_j)\hat{U}_j$,

$$\hat{\mu}_j(t+1) = \mu_j(t) + \alpha_\mu\, q'(\mu_j(t))\, \hat{U}_{j,t}, \qquad \mu(t+1) = \Pi_{\{\mu \geq 0,\ \sum_j \mu_j \leq B\}}\big(\hat{\mu}(t+1)\big),$$

with $q'(\mu) = c\,e^{-c\mu}$. The projection has a closed form via the capped-simplex projection. Two-timescale behavior is obtained by choosing $\alpha_U \gg \alpha_\mu$, so the utilities are accurate before the attention moves.

6.1 Learnable Following Rates with Fixed Actor Policies

This subsection studies the following-rate update alone. With fixed Actors of known utility, projected gradient ascent should move the budget B toward the best Actors.

Controlled test 1: one strictly-best Actor. A single state, three fixed Actors, and reward row $R = [0.9, 0.5, 0.1]$. Each fixed Actor uses a sharp softmax centered on its target action, so the analytical utilities round to $[0.9, 0.5, 0.1]$. We use $B = 1$, $c = 2$, $\alpha_U = 0.10$, $\alpha_\mu = 0.02$, and 4000 steps. The expected behavior is $\hat{U} \to [0.9, 0.5, 0.1]$, μ concentrating on Actor 0, the low-utility Actor 2 falling to $\mu_2 \approx 0$, and the budget binding.

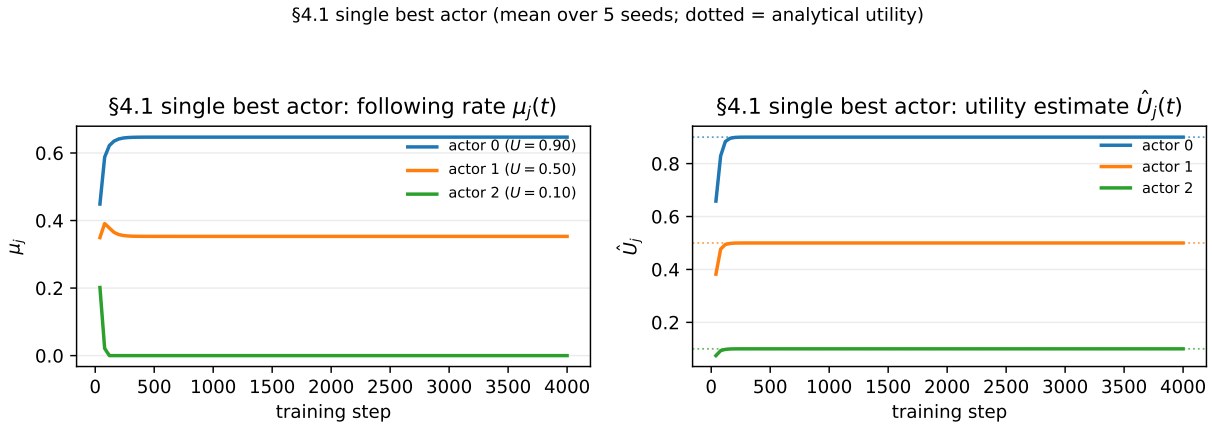


Figure 7: Single-best following-rate result. Left: following rates $\mu_j(t)$ — the high-utility Actor 0 gains budget while the low-utility Actor 2 drops to zero. Right: utility estimates $\hat{U}_j(t)$ match the dotted analytical utilities. Mean over 5 seeds.

Quantity	Actor 0	Actor 1	Actor 2
Analytical utility U_j	0.900	0.500	0.100
Final estimate $\hat{U}_j$	0.900	0.500	0.100
Final following rate μ_j	0.647	0.353	0.000

Table 6: Single-best following-rate test, mean over 5 seeds. The budget binds ($\sum_j \mu_j = 1.000$) and attention is ordered by true utility ($\mu_0 > \mu_1 > \mu_2$).

Controlled test 2: tie. The same setup with $R = [0.9, 0.9, 0.1]$, so Actors 0 and 1 are equally useful and Actor 2 has low utility. The expected behavior is that the budget splits across the tied optimal Actors while Actor 2 loses attention.

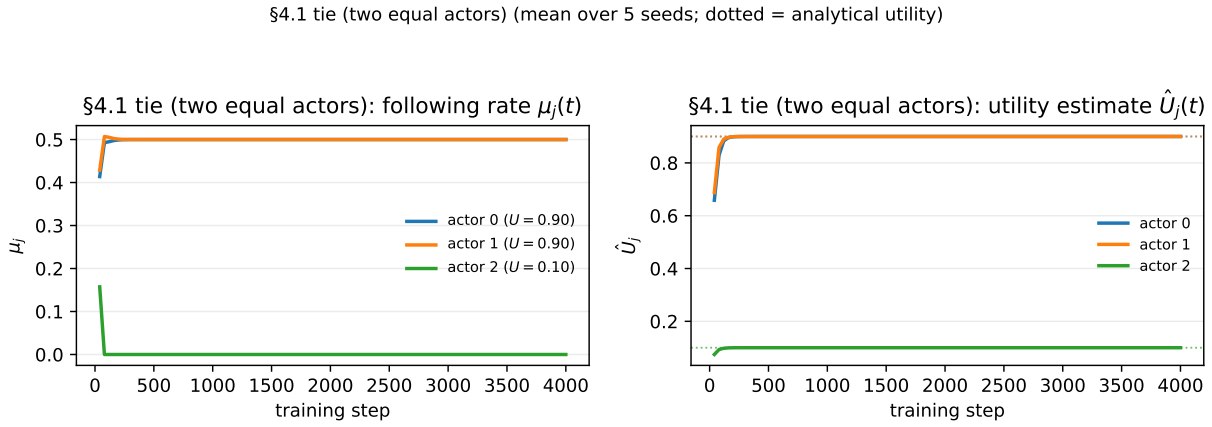


Figure 8: Following-rate tie result. The two equally useful Actors share the budget symmetrically and the low-utility Actor 2 drops to zero. Because the two top Actors are genuinely symmetric, the across-seed mean equals the per-seed behavior. Mean over 5 seeds.

Quantity	Actor 0	Actor 1	Actor 2
Analytical utility U_j	0.900	0.900	0.100
Final estimate $\hat{U}_j$	0.900	0.900	0.100
Final following rate μ_j	0.500	0.500	0.000

Table 7: Following-rate tie test, mean over 5 seeds. Mass leaves the low-utility Actor and splits evenly across the tied optimal set.

Both tests show the learned variables directly: the utility estimates recover the true utilities and the following rates move to the high-utility set under the budget constraint.

6.2 Learnable Following Rates and Actor Policies

Here each Actor also learns. A reported step — one where the Bernoulli draw Bernoulli($q(\mu_j)$) succeeds — drives a REINFORCE update

$$\theta_j \leftarrow \theta_j + \alpha_\theta \, r_t \, \nabla_{\theta_j} \log \pi_{\theta_j}(a_t \mid s_t).$$

The Observer's μ and $\hat{U}$ update as in the fixed-policy experiment.

Setup. Three states (uniform), three actions, three Actors, reward table

$$R = \begin{pmatrix} 1.0 & 0.1 & 0.2 \\ 0.1 & 1.0 & 0.0 \\ 0.0 & 0.2 & 1.0 \end{pmatrix},$$

optimal actions $[0, 1, 2]$, $B = 3$, $c = 2$ (so the initial $\mu = [1, 1, 1]$ gives $q(1) \approx 0.865$, meaning most acted steps are reported), $\alpha_U = 0.20$, $\alpha_\mu = 0.01$, $\alpha_\theta = 0.20$, and 30000 steps. Since every Actor can become optimal and the budget is large enough to fund all of them, the correct fixed point is each Actor's optimal-action probability approaching one, $\hat{U}_j \to 1$, and a symmetric allocation $\mu \to [1, 1, 1]$.

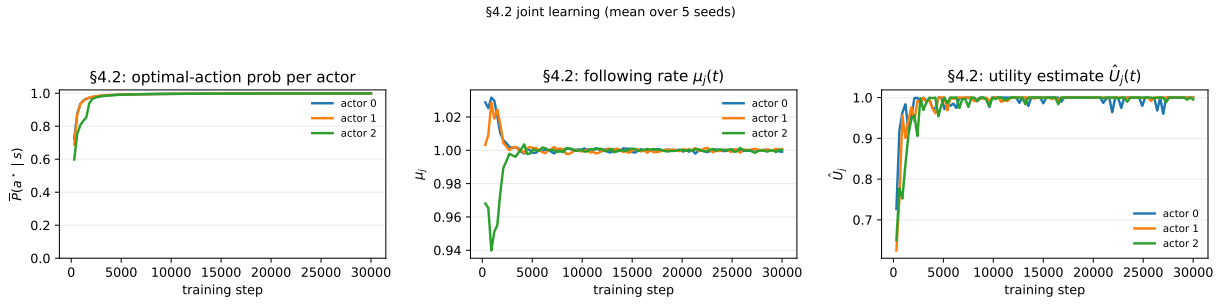


Figure 9: Joint following-rate and Actor-policy learning. Left: per-Actor optimal-action probability climbs to ≈ 0.999. Middle: following rates $\mu_j(t)$ remain near the feasible symmetric allocation. Right: utility estimates $\hat{U}_j(t)$ rise toward the per-Actor ceiling of one. Mean over 5 seeds.

Quantity	Actor 0	Actor 1	Actor 2
Final optimal-action prob.	0.999	0.999	0.999
Final following rate μ_j	0.999	1.001	1.000

Table 8: Joint following-rate and Actor-policy learning, mean over 5 seeds. All Actors reach near-deterministic optimal policies and the budget stays feasible ($\sum_j \mu_j = 3.000$).

In this controlled setup, joint learning is stable with the two-timescale choice ($\alpha_U \gg \alpha_\mu$): the fast utility estimate tracks the improving Actors, and the slow following-rate update keeps μ feasible throughout. The figures show the learned quantities directly: policies improve, utilities rise, attention stays within budget, and end-of-run reward is high.

7 Group-Level Interactions: Imperfect Reward

This section is the paper's group-level imperfect-reward extension of the previous one. Instead of a single Observer watching N Actors, there are now N Observers $O_1, \ldots, O_N$ and N Actors $A_1, \ldots, A_N$. Every Actor still sees the state perfectly, but each Observer O_i now has its *own* reward function R_i: when the active Actor performs an action, Observer i perceives reward $R_i(s_t, a_t)$. Because the R_i differ, two Observers may value the same Actor differently, and each Observer maintains its own budgeted following-rate vector $\mu_i = (\mu_{i,1}, \ldots, \mu_{i,N})$ and utility estimate $\hat{U}_{i,j}$.

Setup conventions. As before, the controlled fixed-policy test uses a single state, three actions, scalar state-ID inputs mapped through a learnable embedding, and hard-coded per-Observer reward tables; one Actor $j_t \sim p$ acts per step with p uniform. Each Observer maintains its own capped-simplex following rates with budget B and reporting probability $q(\mu_{i,j}) = 1 - e^{-c\,\mu_{i,j}}$.

Updates. When Actor j_t acts, every Observer updates its utility estimate for that Actor on the fast timescale using its own perceived reward,

$$\hat{U}_{i,j_t,t+1} = \hat{U}_{i,j_t,t} + \alpha_U \big(R_i(s_t, a_t) - \hat{U}_{i,j_t,t}\big),$$

and updates its following rates by the same projected gradient ascent as the single-Observer case. In the joint-learning setting, Actor j additionally learns from an Observer-aggregated reward (paper Eq. 29),

$$r_{\text{eff}}(j) = \sum_{i=1}^{N} q(\mu_{i,j})\, R_i(s_t, a_t), \qquad \theta_j \leftarrow \theta_j + \alpha_\theta\, r_{\text{eff}}(j)\, \nabla_{\theta_j} \log \pi_{\theta_j}(a_t \mid s_t),$$

so an Actor's learning signal reflects both the reward for its action and how often Observers report that reward.

7.1 Multiple Observers and Actors with Learnable Following Rates

This subsection studies whether each Observer, using its own imperfect reward function, reallocates its budget toward the Actors *it* values most while Actor policies are fixed.

Controlled test: assortative reward. A single state, three fixed Actors (Actor j pinned to action j), and three Observers with diagonal reward tables $R_0 = [0.9, 0.4, 0.1]$, $R_1 = [0.1, 0.9, 0.4]$,

$R_2 = [0.4, 0.1, 0.9]$, so Observer i most values Actor i. We use $B = 2$, $c = 1$, $\alpha_U = 0.10$, $\alpha_\mu = 0.05$, and 8000 steps. With these utilities the constrained optimum is the closed-form KKT allocation $\mu = [1.405, 0.595, 0]$ (reordered per Observer): the highest-utility Actor receives the most budget and the worst Actor drops to zero.

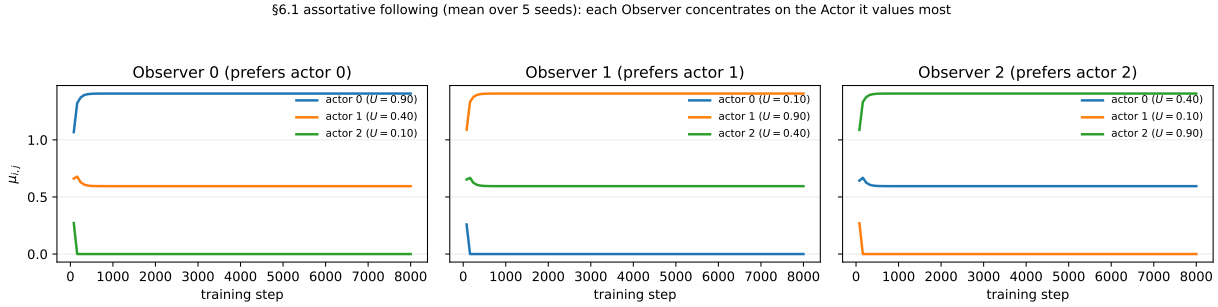


Figure 10: Assortative following with heterogeneous Observers. Each panel shows one Observer's following rates $\mu_{i,j}(t)$; every Observer concentrates its budget on the Actor it values most, so the three Observers end with different Actor preferences according to their own reward functions. Mean over 5 seeds.

Final following rate $\mu_{i,j}$	Actor 0	Actor 1	Actor 2
Observer 0 ($U = [0.9, 0.4, 0.1]$)	1.405	0.595	0.000
Observer 1 ($U = [0.1, 0.9, 0.4]$)	0.000	1.405	0.595
Observer 2 ($U = [0.4, 0.1, 0.9]$)	0.595	0.000	1.405

Table 9: Assortative following test, mean over 5 seeds. Each Observer's budget binds ($\sum_j \mu_{i,j} = 2.000$) and concentrates on its highest-utility Actor, matching the analytical KKT allocation.

The empirical following rates match the analytical allocation, and the three Observers move toward three different Actors. This is the group-level experiment: different reward functions produce different budget allocations.

7.2 Multiple Observers and Actors with Joint Learning

Here each Actor also learns, using the Observer-aggregated reward $r_{\text{eff}}(j) = \sum_i q(\mu_{i,j}) R_i$ defined above.

Setup. Three states (uniform), three actions, three Actors, and three Observers sharing the diagonal reward table

$$R = \begin{pmatrix} 1.0 & 0.1 & 0.2 \\ 0.1 & 1.0 & 0.0 \\ 0.0 & 0.2 & 1.0 \end{pmatrix},$$

optimal actions $[0, 1, 2]$, $B = 3$, $c = 2$, $\alpha_U = 0.20$, $\alpha_\mu = 0.01$, $\alpha_\theta = 0.10$, and 30000 steps. Because all Actors can become optimal, they become equally valuable to every Observer, so the correct fixed point is each Actor's optimal-action probability approaching one and a symmetric allocation $\mu_i \to [1, 1, 1]$ for every Observer.

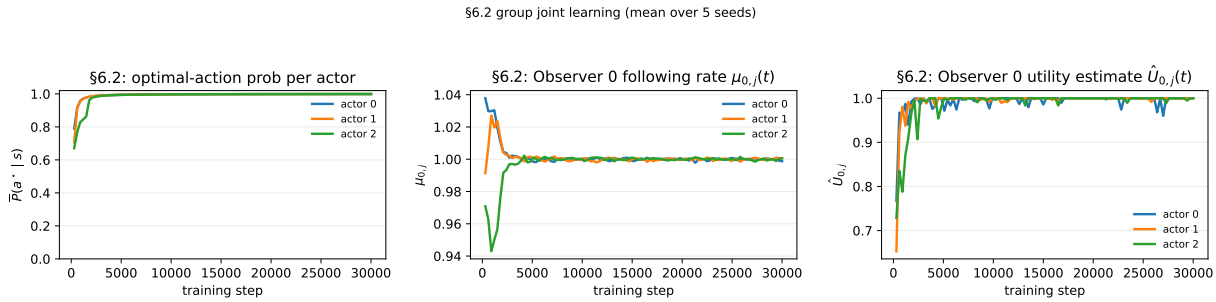


Figure 11: Group-level joint learning. Left: per-Actor optimal-action probability climbs to ≈ 0.999 under the Observer-aggregated reward. Middle: Observer 0's following rates $\mu_{0,j}(t)$ stay near the feasible symmetric allocation. Right: Observer 0's utility estimates $\hat{U}_{0,j}(t)$ rise toward the per-Actor ceiling of one. Mean over 5 seeds.

Quantity	Actor 0	Actor 1	Actor 2
Final optimal-action prob.	0.999	0.999	0.999
Final following rate $\mu_{i,j}$ (any i)	1.000	1.000	1.000

Table 10: Group-level joint learning, mean over 5 seeds. All Actors reach near-deterministic optimal policies under the aggregated reward, and every Observer's budget stays feasible ($\sum_j \mu_{i,j} = 3.000$).

In this controlled setup, joint group learning is stable with the two-timescale choice. The aggregated reward $\sum_i q(\mu_{i,j})R_i$ trains the Actors, and every Observer's following rates stay within budget.

8 Imperfect State with Multiple Actors and One Observer

One Observer chooses a message source, receives one scalar message, and acts. The standard setup has three Actors: a zero-input source, a partial source that observes the first state coordinate only, and a full source that observes every coordinate. The $K = 5$ ladder uses five sources by adding two further partial sources with the same restricted input. They make the selection problem less favorable to the full source without changing the state or reward function.

The learned selection distribution is λ, and $P(\text{full})$ is its probability for the full Actor. A fixed correct selection distribution always chooses that source. The seven experiments below hold different subsets of ϕ, w, and θ fixed. Where compression is fixed, the full source sends the coordinate sum and the other sources retain their restricted information. Where policy is fixed, it is the non-learned reference policy from the Method section.

8.1 Learning Compression Only

Only the selected full Actor changes its message weights ϕ. Selection is fixed to the full source and the action rule is fixed. This asks whether reward feedback alone can make the full source send a useful scalar message.

8.2 Learning Selection Only

Only the source-selection weights w are learned. Compression is fixed to the correct scalar sum and the action rule is fixed. This asks whether the Observer moves its selection probability toward the informative source.

8.3 Learning Policy Only

Only the adaptive policy θ is learned. Selection is fixed to the full source and its message is the correct scalar sum. This is the direct policy-learning control for the one-Observer setting.

8.4 Learning Compression and Selection

Compression ϕ and selection weights w are learned, while the action rule remains fixed. The result separates the information-source choice from learned action probabilities.

8.5 Learning Selection and Policy

Selection weights w and policy θ are learned while the full source keeps the correct scalar message. This keeps the message stable and tests selection and action learning together.

8.6 Learning Compression and Policy

Compression ϕ and policy θ are learned while selection is fixed to the full source. This retains the continuous message-policy feedback path while removing source-selection variation.

8.7 Learning Compression, Selection, and Policy

The full experiment learns the message weights ϕ, selection weights w, and policy parameters θ together. The initial compression is positive random; the adaptive policy parameters are randomly initialized as described in the Method section. No state-action targets are used during training.

Learned object	Reward	P(full)	Lowest correct	Regions below 0.80	Stable seeds	Parameters stable
Compression	0.977	1.000	0.965	0.00	5/5	5/5
Selection	0.965	0.995	0.965	0.00	5/5	5/5
Policy	1.000	1.000	1.000	0.00	5/5	5/5
Compression and selection	0.972	0.993	0.965	0.00	5/5	5/5
Selection and policy	0.995	0.993	1.000	0.00	5/5	5/5
Compression and policy	1.000	1.000	1.000	0.00	5/5	5/5
All three	0.995	0.993	1.000	0.00	5/5	5/5

Table 11: One-Observer, five-Actor ladder at $K = A = 5$. All rows use 300k steps, $\alpha_\phi = \alpha_w = \alpha_\theta = 0.001$, five seeds, and positive-random compression initialization. A learned selection or policy uses its 0.20 uniform floor; the corresponding floor is zero when that object is fixed. A fixed object is not included in its active learning rate.

Figure file `rng_clean_section8_k5_c5_ladder_curves` is not in the repository yet.

Figure 12: One-Observer, five-Actor learning curves for the seven $K = A = 5$ experiments. The curves track reward, full-source selection probability, correct-action probability, and full-message mean absolute error relative to the state index.

The first scale increase uses three Actors and one Observer. For $K = 16, A = 16$, the run length is 800k steps. For $K = 32, A = 16$, it is 1.6 million steps and each action represents two neighboring state values. In both tables, all active learning rates are 0.001. A learned selection or policy uses its 0.20 exploration floor, while a fixed object uses no floor. The five-seed results satisfy the final-window stability criterion.

	$K = A = 16$, 800k steps				$K = 32, A = 16$, 1.6M steps			
Learned object	Reward	P(full)	Lowest correct	Seeds	Reward	P(full)	Lowest correct	Seeds
Compression	0.966	1.000	0.965	5/5	0.966	1.000	0.965	5/5
Selection	0.962	0.999	0.965	5/5	0.875	0.999	0.879	5/5
Policy	1.000	1.000	1.000	5/5	1.000	1.000	1.000	5/5
Compression and selection	0.965	0.998	0.965	5/5	0.965	0.999	0.965	5/5
Selection and policy	0.999	0.999	1.000	5/5	0.999	0.999	1.000	5/5
Compression and policy	0.972	1.000	0.800	4/5	0.986	1.000	0.800	4/5
All three	0.999	0.999	1.000	5/5	0.999	0.999	1.000	5/5

Table 12: One-Observer scale results with independent random streams. "Seeds" gives the number of seeds with no state-action region below 0.80. The compression-and-policy row is seed-sensitive at both sizes: four of five seeds meet the internal criterion, although its reward and selection probability remain high.

Figure file `rng_clean_section8_k16_full_curves` is not in the repository yet.

Figure 13: Full one-Observer experiment at $K = A = 16$. The four panels show the learned expected reward, the lowest correct-action probability, the movement of the full-source messages as mean absolute error relative to the state index, and the full-source selection probability.

At $K = 64, A = 16$, the full three-object experiment was run for 3.2 million steps with one completed seed, positive-unit compression initialization (weights sampled from $[0.5, 1.5]$), an learning rate of 0.001, and both exploration floors at 0.20. It reached reward 0.9997, P(full) = 0.9997, lowest correct-action probability 1.000, and no regions below 0.80. Its final reward range was $5.1 \cdot 10^{-5}$; message and policy movements were zero at the recorded precision. This full three-object row remains a stress demonstration. An independent five-seed compression-and-policy rerun at the same size, with selection fixed to the full source, reaches mean reward 1.000, lowest correct-action probability 1.000, and no regions below 0.80 in all five seeds.

Figure file `rng_clean_section8_k64_a16_full_positive_unit_seed0_curves` is not in the repository yet.

Figure 14: One completed full one-Observer stress run at $K = 64, A = 16$. The action mapping groups four consecutive states per action. The final checkpoints show stable reward, policy probabilities, full-source selection, and full-message mean absolute error relative to the state index.

The fixed-policy rows have expected reward below one because the reference policy has finite action sharpness. Their correct action is still the most probable in every state. In contrast, the rows with a learned policy show whether the adaptive scalar-message policy itself has formed the required action regions. The full experiments learn all three objects in every five-seed result through $K = 32, A = 16$, with the stated $K = 64, A = 16$ stress demonstration.

9 Imperfect State with Multiple Actors and Multiple Observers

This is the group imperfect-state setting. Each Observer independently selects an Actor, receives that source's scalar message, and chooses an action at every environment step. The standard configuration has three Actors and three Observers. The sources are the same zero, partial, and full sources used in the preceding section. The expected reward, action probabilities, and $P(\text{full})$ are averaged over the Observers; the lowest correct-action probability is taken over all states and Observers.

The seven subcases are the same as in the one-Observer setting. They are listed separately below because the group loop adds several simultaneous selection and action updates each step.

9.1 Learning Compression Only

Only ϕ is learned. Every Observer is fixed to the full source and uses the reference action policy.

9.2 Learning Selection Only

Only the selection weights w are learned. The full source sends the correct scalar sum and the action rule is fixed.

9.3 Learning Policy Only

Only the adaptive policy parameters θ are learned. Every Observer uses the fixed full source and receives its correct scalar message.

9.4 Learning Compression and Selection

Compression ϕ and selection weights w are learned with a fixed action rule.

9.5 Learning Selection and Policy

Selection weights w and policies θ are learned while compression is fixed to the correct scalar sum.

9.6 Learning Compression and Policy

Compression ϕ and policies θ are learned while each Observer is fixed to the full Actor.

9.7 Learning Compression, Selection, and Policy

The full group experiment learns ϕ, w, and θ together. All adaptive-policy parameters remain source-specific; a selected source has its own feature centers, widths, affine map, and action layer.

	$K = A = 3$, 200k steps						$K = A = 5$, 300k steps					
Learned object	Reward	P(full)	Lowest	Below .80	Seeds	Stable	Reward	P(full)	Lowest	Below .80	Seeds	Stable
Compression	.987	1.000	.965	0	5/5	5/5	.977	.996	.965	0	5/5	5/5
Selection	.969	.990	.965	0	5/5	5/5	.966	.996	.965	0	5/5	5/5
Policy	1.000	1.000	1.000	0	5/5	5/5	1.000	1.000	1.000	0	5/5	5/5
Compression and selection	.981	.988	.965	0	5/5	5/5	.973	.995	.965	0	5/5	5/5
Selection and policy	.994	.988	1.000	0	5/5	5/5	.996	.995	1.000	0	5/5	5/5
Compression and policy	1.000	1.000	1.000	0	5/5	5/5	1.000	1.000	1.000	0	5/5	5/5
All three	.994	.988	1.000	0	5/5	5/5	.996	.995	1.000	0	5/5	5/5

Table 13: Three-Actor, three-Observer learning ladders with independent random streams. All rows use $\alpha_\phi = \alpha_w = \alpha_\theta = 0.001$, positive-random compression initialization, and five seeds. A learned selection or policy uses its 0.20 uniform floor, while the floor is zero when that object is fixed. "Lowest" is the lowest correct-action probability.

Learned object	Reward	P(full)	Lowest correct	Regions below 0.80	Seeds	Stable
Compression	0.971	1.000	0.965	0.00	5/5	5/5
Selection	0.964	0.997	0.965	0.00	5/5	5/5
Policy	1.000	1.000	1.000	0.00	5/5	5/5
Compression and selection	0.968	0.997	0.965	0.00	5/5	5/5
Selection and policy	0.997	0.997	1.000	0.00	5/5	5/5
Compression and policy	1.000	1.000	1.000	0.00	5/5	5/5
All three	0.997	0.997	1.000	0.00	5/5	5/5

Table 14: Three-Actor, three-Observer ladder at $K = A = 8$. The run uses 400k steps and active learning rate 0.001 for every learned object. A learned selection or policy uses its 0.20 uniform floor. Every reported seed is stable in the final window.

Figure file `rng_clean_section9_k8_full_curves` is not in the repository yet.

Figure 15: Full three-Actor, three-Observer experiment at $K = A = 8$. Expected reward, full-source selection probability, correct-action probability, and full-message mean absolute error relative to the state index settle during the post-anneal window.

The full $K = A = 16$ group run used 800k steps and the same learning rate and floors. Four of five seeds have reward 0.999, lowest correct-action probability 1.000, P(full) = 0.999, and no regions below 0.80. The remaining seed settles to a lower action mapping, giving the five-seed mean reward 0.994, mean lowest correct probability 0.800, and mean 0.20 regions below 0.80. This sensitivity is recorded here rather than hidden by the mean reward.

For grouped actions, the full three-object run at $K = 32, A = 16$ uses 1.6 million steps and reaches mean reward 0.999, P(full) = 0.999, lowest correct probability 1.000, and no regions below 0.80 in all five seeds. At $K = 64, A = 16$, all seven subcases were run for 3.2 million steps in one seed with positive-unit compression initialization. The full row reaches reward 0.9997, P(full) = 0.9997, and

lowest correct probability 1.000, with no missed regions. The $K = 64$ result is a one-seed stress demonstration.

Setting	Steps	Seeds	Reward	P(full)	Lowest correct	Regions below 0.80
Full, $K = A = 16$	800k	4/5 successful	0.999	0.999	1.000	0.00
Full, $K = 32, A = 16$	1.6M	5/5	0.999	0.999	1.000	0.00
Full, $K = 64, A = 16$	3.2M	1 successful	0.9997	0.9997	1.000	0.00

Table 15: Larger group full experiments. All use three Actors, three Observers, $\alpha_\phi = \alpha_w = \alpha_\theta = 0.001$, and a 0.20 uniform floor for each learned selection or policy. The $K = 64$ run uses positive-unit compression initialization; the other rows use positive-random initialization.

Figure file `rng_clean_section9_k64_a16_full_seed0_curves` is not in the repository yet.

Figure 16: One completed three-Actor, three-Observer full stress run at $K = 64, A = 16$. The policy groups four neighboring state values per action. The curves provide the final-window stability record for reward, action probabilities, full-message mean absolute error, and full-source selection.

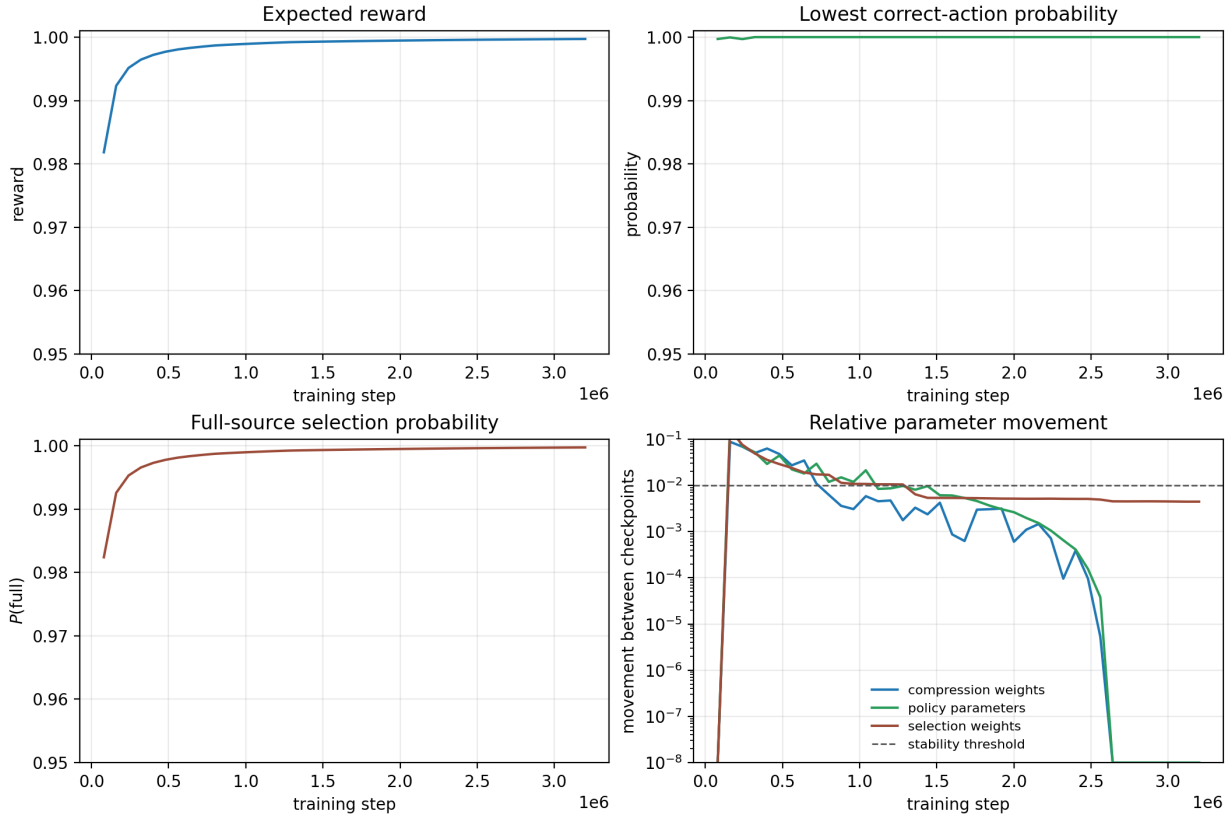


Figure 17: Learned-parameter convergence in the same $K = 64, A = 16$ group run. The lower-right panel is the relative change in the compression weights, policy parameters, and selection weights between saved checkpoints. The dashed line is the 0.01 stability threshold used in this report.

10 Combined Imperfect State and Imperfect Reward

The combined setting joins the two information problems from the paper. Agents take environmental actions from selected peer messages, while reward Observers allocate following effort under a rate budget and report evaluative feedback. The four learned updates are compression ϕ, information-source selection w (equivalently λ), environmental policy θ, and reward-following rates μ (with reporting probabilities $q(\mu)$). This section keeps all four updates in view; it does not reduce the story to a two-parameter sketch, and it does not introduce a critic or influencer extension.

Terminology follows the paper roles. On the imperfect-reward side there is one Observer (or several Observers) following many Actors. On the imperfect-state side there are multiple Actors, each sending a compressed state message, and one or more Observers that select a message source and act.

Earlier combined numbers were produced before the independent random-stream correction in the Method section, so they are not reported here. All numbers below use the corrected trainer, which separates environment, action, selection, and reporting streams.

10.1 Color-coded status

The 12 August meeting asked for a color-coded draft of this section, not only a success table. Black is a claim that can be taken as it stands on the stated task. Green is a problem that was found and then changed. Red is still open and should be read as a pressure point: it remains informative even when the run fails. Table 16 is only the legend. The three subsections after it give the setup, policy head, initialization, and evidence for each color.

Status	How to read it
Black	Unchanged and trustworthy on the monotone shared-reward combined task after the random-stream correction.
Green	A concrete failure was isolated, one setting was changed, and the same ladder then works. Least urgent to reread once the change is noted.
Red	Uncertain, structurally limited, or still broken. Keep these in view; do not treat a missing or failed cell as a hole in the black claims.

Table 16: Color legend for Section 10. Black is the result. Green is a fixed defect. Red is a pressure point on this task or a later task.

Black: four-quantity isolation.
The runner can freeze or learn every subset of $\{\phi, w, \theta, \mu\}$. Table 17 is the complete 16-rung matrix at $K = A = N = L = 3$, three seeds, 120k steps. Every learned-θ rung uses `LocalMessageAdaptiveRBFPolicy`: random Gaussian centers ($\sigma = 1$), widths started from a small gap, a learned message affine map, and a random linear action head. Centers are not placed on integer state IDs and are not copied from $a^\star(k)$. When θ is frozen, the control is the known distance decoder $\ell_a = -\beta(z - c_a)^2$ with c_a the mean state index of action a. That control is never used as a warm start for a learned policy. Replay, supervised labels, and target-shaped policy initialization are off. Every rung has reward at least 0.970, weakest correct-action probability at least 0.948, and no missed region. The claim is isolation on this small combined loop, not robustness at large K.

Black: full learning through $K = 16$ and 5×5.
On the same monotone grouped target and shared Observer tables, all four quantities work after the stream correction at $K \in \{3, 5, 8, 16\}$ with three acting agents and three reward Observers (Table 18). The common recipe is shared step size 0.001, utility step 0.05, $\epsilon_w = \epsilon_a = 0.20$, and $\epsilon_\mu = 0.20$ once $K \geq 5$. The $K = 3$ ladder still uses $\epsilon_\mu = 0.02$. At $K = 16$, $A = 8$, 640k steps, all-four reward is 0.994 and the weakest correct-action probability is 0.983. The same recipe also works at five agents and five reward Observers (reward 0.990, weakest correct 0.946; Table 19). $K = 32$ is still black after one seed is continued to 1.28M (all three seeds then have no missed region). It is not a claim that $K = 64$ all-four or 8×8 work.

Black: the separate tracks remain the baselines.
Imperfect-state alone and imperfect-reward alone, including their many-to-many versions, are the objects this high-budget combined probe is supposed to recover. Those sections are unchanged by the combined reruns. Combined success is read against that baseline, not against a new architecture.

Green: independent random streams.
Earlier combined numbers used a shared environment/action stream. After the Method-section correction the trainer separates environment, action, selection, and reporting streams. Every number in this section is from the corrected trainer. Pre-correction combined tables are discarded and are not mixed with the rows below.

Green: learned θ is random adaptive RBF.
The pressure point named in the meeting is learning compression together with a non-linear policy head, not a hard-coded distance decoder. The reruns therefore put random adaptive RBF on every learned-θ rung. The θ-only and ϕ, θ rows at $K = 3$ start near random reward and reach 0.998. That is evidence that the *action* map is not written into the initialization. The head still sums the local observation; that input is in the setup below, not a separate red cell.

Green: $K = 5$ reporting starvation.
At $\epsilon_\mu = 0.02$, every combination through ϕ, w, θ stays high, but w, θ, μ falls to reward 0.820 (weakest correct 0.188) and all four fall to 0.872 (weakest correct 0.440). Failed seeds show one agent's report probability near 0.007 while the others remain near 0.45, so that agent loses reward feedback and keeps a diffuse source distribution. Raising only the uniform report floor to $\epsilon_\mu = 0.20$ restores both hard rows to reward 0.992 in all three seeds. The failure is reward-feedback starvation, not adaptive-RBF capacity. The same floor is then kept for $K \geq 5$. This is a feedback-floor fix, not a new module.

Red: $K = 64$ all-four is a ϕ, θ interaction, not a selection miss.
At $K = 64$, $A = 8$, one seed, 1.92M steps, θ-only reaches reward 0.986 and ϕ, θ reaches 0.993. All four reach reward 0.964, but the weakest correct-action probability is 0.010 and seven regions are missed, while $P(\text{informative}) = 0.999$. The source is already found; the remaining issue is the ϕ, θ timescale under sparse reporting.

Red: 8×8 is a selection failure.
At $K = 16$, $A = 8$, five agents and five reward Observers work. Eight and eight do not: reward 0.926, $P(\text{informative}) = 0.233$, nine missed regions. Raising ϵ_w to 0.50 and continuing to 960k leaves selection worse (0.186).

10.2 Setup, architecture, and initialization

There are K equally likely states. State k is the cumulative binary vector with k leading ones. Acting agent i sees a local prefix of length $h = \max(1, \lfloor (K-1)/2 \rfloor)$ and selects a peer message m_j. Two peers hold the complementary suffix, so an informative source exists. The policy input is the concatenation (o_i, m_j).

When θ is learned, the policy is `LocalMessageAdaptiveRBFPolicy`. It first forms one scalar,

$$z_i = c_o \sum_r o_{i,r} + c_m m_j + b,$$

rescales z_i to an approximate state-index range, and then applies a random-initialized adaptive radial-basis-function head: learned centers, widths, a message affine map, and a linear action layer followed by softmax. The construction is differentiable in m_j, so the compression update can use $\partial \log \pi / \partial m$. Centers are not placed on integer state IDs and are not copied from the target action map. When θ is frozen, the control policy is the known distance decoder $\ell_a = -\beta(z_i - c_a)^2$ with c_a equal to the mean state index of action a. That control is never used as a warm start for a learned policy.

The target action is grouped and monotone,

$$a^\star(k) = \min\{A - 1, \lfloor kA/K \rfloor\}.$$

A correct action receives 1.0 and any other action receives -0.1. Unless a diagnostic below says otherwise, every reward Observer uses this same table. Agent i then learns from

$$\widetilde{R}_i = \sum_{\ell=1}^{L} q(\mu_{\ell,i}) R_\ell(s, a_i), \qquad q(\mu) = 1 - e^{-c\mu}.$$

With identical tables this factors as a positive scale times the true reward. Report probabilities are the expected aggregate, not a Bernoulli gate. Each Observer starts at $q(\mu_{\ell,i}) = 1/L$, so the initial report sum is one.

The common full-learning recipe after the stream correction is shared step size 0.001 for ϕ, w, θ, μ, utility step 0.05, $\epsilon_w = \epsilon_a = 0.20$, and $\epsilon_\mu = 0.20$ once $K \geq 5$. The $K = 3$ ladder below still uses $\epsilon_\mu = 0.02$. Replay, supervised labels, and target-shaped policy initialization are off.

10.3 Sixteen-rung ladder at $K = A = 3$

Table 17 is the complete isolation matrix on the monotone shared-reward task. Three seeds, 120k steps. Every θ rung uses the random adaptive-RBF head.

Learned	Reward	Weakest $P(a^\star)$	P(informative)	Missed regions
none	0.974	0.965	1.000	0
ϕ	0.981	0.962	1.000	0
w	0.970	0.949	0.989	0
θ	0.998	0.998	1.000	0
μ	0.974	0.965	1.000	0
ϕ, w	0.976	0.951	0.986	0
ϕ, θ	0.998	0.997	1.000	0
ϕ, μ	0.981	0.962	1.000	0
w, θ	0.993	0.977	0.986	0
w, μ	0.970	0.948	0.988	0
θ, μ	0.998	0.998	1.000	0
ϕ, w, θ	0.993	0.977	0.986	0
ϕ, w, μ	0.976	0.951	0.985	0
ϕ, θ, μ	0.998	0.997	1.000	0
w, θ, μ	0.993	0.976	0.985	0
ϕ, w, θ, μ	0.993	0.976	0.985	0

Table 17: RNG-clean combined ladder at $K = A = N = L = 3$. Three-seed means, 120k steps, adaptive-RBF θ, shared reward tables, monotone targets. Source: `docs/rng_clean_section10_adaptive_rbf_results.md`.

The θ-only / ϕ, θ rise from random reward is the green RBF-init claim above. The μ-only row matching the all-fixed row is the shared-table degeneracy recorded in Table 17, not a separate red cell.

10.4 State and population scaling

Table 18 keeps three acting agents and three reward Observers and increases K. Table 19 keeps $K = 16$, $A = 8$ and increases the population. Both tables use the adaptive-RBF recipe with $\epsilon_\mu = 0.20$.

K	A	Learned	Seeds	Steps	Reward	Weakest $P(a^\star)$	P(inf.)	Missed
3	3	all four	3	120k	0.993	0.976	0.985	0
5	5	all four	3	180k	0.992	0.979	0.991	0
8	4	all four	3	240k	0.995	0.983	0.994	0
16	8	all four	3	640k	0.994	0.983	0.998	0
32	8	all four	3	640k–1.28M	0.984–0.993	0.845–0.946	≥ 0.998	0
64	8	θ	1	1.92M	0.986	0.916	1.000	0
64	8	ϕ, θ	1	1.92M	0.993	0.900	1.000	0
64	8	all four	1	1.92M	0.964	0.010	0.999	7

Table 18: State scaling on the monotone shared-reward task. The $K = 32$ range is three seeds; one seed needed continuation to 1.28M. The $K = 64$ rows are one seed after a 640k settling phase at step size 0.00025.

N	L	Steps	ϵ_w	Reward	Weakest $P(a^\star)$	P(inf.)	Missed
3	3	640k	0.20	0.994	0.983	0.998	0
5	5	640k	0.20	0.990	0.946	0.978	0
8	8	640k	0.20	0.926	0.010	0.233	9
8	8	960k	0.50	0.946	0.005	0.186	7

Table 19: Population scaling at $K = 16$, $A = 8$, all four quantities learned. The 5×5 row works under the common recipe. The 8×8 row fails in source selection; raising the selection floor does not fix it.

Figure file `rng_clean_section10_scaling_summary` is not in the repository yet.

Figure 18: State and population scaling for the RNG-clean combined recipe. The $K = 64$ all-four point and the 8×8 point are the present boundaries on the monotone shared-reward task.

The $K = 5$ starvation, $K = 64$ all-four cell, and 8×8 selection cell are the green and red items above; the tables only record the numbers. Raw w and μ keep moving under a constant step size even when reward and displayed probabilities are high. The strict all-parameter convergence flag is therefore often false on successful rows. Reward, correct-action probabilities, selection probabilities, and report probabilities are the quantities used to mark a row as working.

11 Discussion

The imperfect-reward results above concern reward feedback and following allocations. The imperfect-state results concern a different chain: an Actor sends a scalar message, an Observer chooses a source, and the Observer selects an action from the message. The two sets of experiments share the paper's reward-gradient framework but have different learned objects.

The post-correction one-to-one experiments show that scalar-message compression, policy learning, and their joint update can all work through eight states and actions in the five-seed table, and through $K = A = 16$ on the three-seed diagonal ladder. From $K = A = 32$ onward the same ladder remains near the fixed-sum decoder when only ϕ is learned, but θ-only and joint ϕ, θ fall together and track the adaptive-RBF oracle ceiling Y_{RBF}. That identifies a policy-head limit under the hard $A = K$ mapping rather than a failure of message learning. A paper-only softmax/SGD contrast collapses earlier on the same diagonal.

In the one-Observer, multi-Actor imperfect-state setting, all seven fixed-and-learned combinations work in five seeds through $K = 32, A = 16$. The full three-object result reaches reward 0.999 with nearly unit full-source selection probability. An independent five-seed compression-and-policy rerun at $K = 64, A = 16$ reaches reward 1.000 with lowest correct-action probability 1.000. The corresponding group setting works in five seeds through $K = A = 8$. At $K = A = 16$, four of five full group seeds reach the same high-reward behavior; the fifth settled to a weaker mapping. The larger $K = 32, A = 16$ and $K = 64, A = 16$ full three-object rows remain stress demonstrations when only one seed is reported.

The current adaptive radial-basis-function policy is important for this comparison. It keeps the scalar message required by the state setup, but it gives the action policy several smooth local features of that message. The feature centers, widths, message affine map, and action-layer weights are all learned from sampled reward. This makes it possible for different scalar message regions to support different actions without converting the message into a one-hot state label. The diagonal ladder shows the remaining limit: when every state needs its own action, the same head can still fail even with oracle messages.

Reward alone is not sufficient to interpret an imperfect-state result. The tables and figures therefore report correct-action probability, full-source selection probability, scalar-message movement, and final parameter movement alongside expected reward. These quantities distinguish a successful action policy from a run that receives good messages but does not use them, and a useful message source from one that is not selected often enough.

The combined section is now filled on the post-correction trainer. On the same monotone shared-reward geometry as the imperfect-state track, random-initialized adaptive-RBF policies learn all four quantities through $K = 16$ and through a 5×5 population. The remaining red cells on that task are the $K = 64$ all-four row and the 8×8 selection row. No critic or influencer extension is part of the present submission cycle.

A Additional Experiments

The appendix contains supporting experiments that are useful for interpreting the main tables: following-rate timescales, tie cases, reward-baseline variants, scaling experiments, and extra con-

vergence plots.

Figure 19 compares three following-rate update schedules on the same single-best controlled setup. The following-rate step is held fixed at $\alpha_\mu = 0.02$, while the utility-estimate rate is varied. When α_U is larger, $\hat{U}$ reaches the true utilities before the following rates have moved far. When α_U is equal to or smaller than α_μ, the same simple problem still converges, but the utility estimate lags and the following-rate movement is correspondingly slower. This supports the two-timescale convention used in the imperfect-reward experiments: the utility estimate should track quickly, while μ moves more slowly under the budget projection.

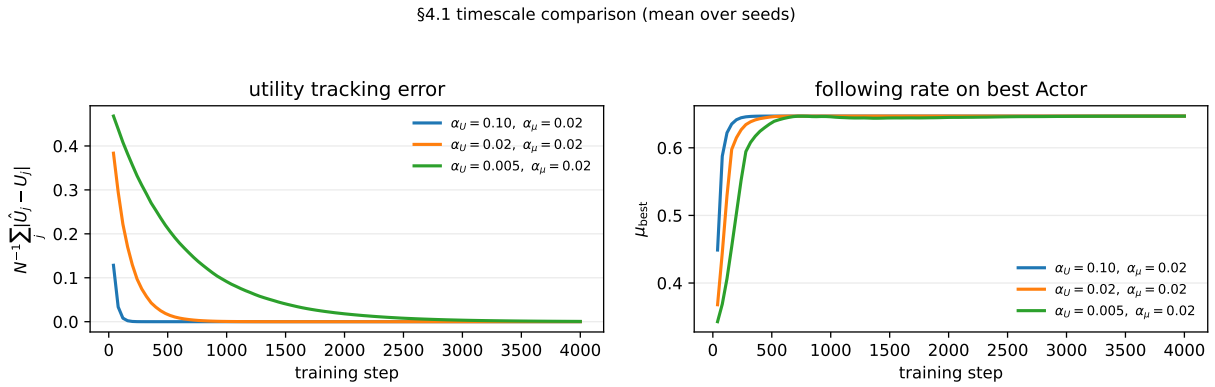


Figure 19: Following-rate timescale comparison. The left panel shows mean utility-tracking error, and the right panel shows the following rate on the best Actor. Larger α_U produces faster utility tracking before the slower following-rate update reallocates the attention budget.

Figure 20 shows tie cases. In the action tie, actions 0 and 1 both have reward 1.0, while action 2 has reward 0.0. The exact split between actions 0 and 1 is seed-dependent, but the low-reward action loses probability. In the Observer tie, two Observers provide the same useful $x_1 + x_2$ message. The probability mass moves to the useful set, even though the split between the two useful Observers can depend on stochastic history.

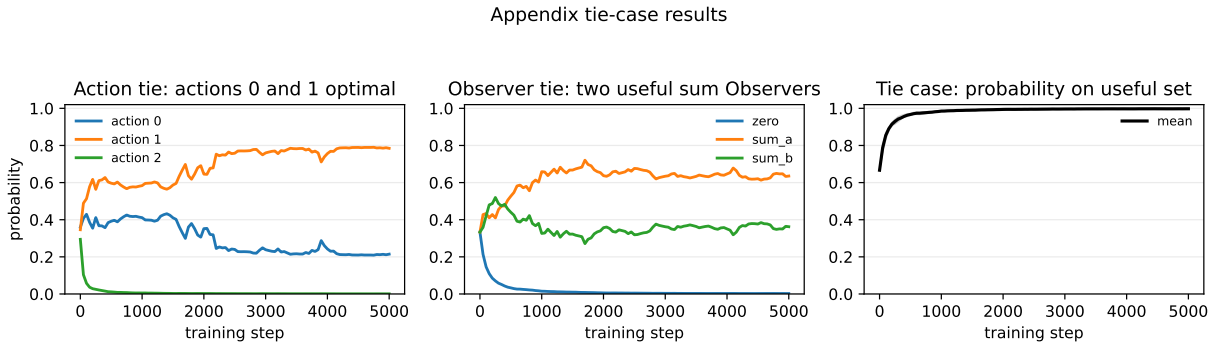


Figure 20: Tie-case results. Tied optimal choices may split unevenly; the required behavior is that probability leaves the lower-value alternatives and concentrates on the optimal set.

Raw Reward and Advantage Variant

The paper's raw-reward selection update uses the raw reward multiplier

$$-r_t \log \lambda_{k_t}.$$

We keep this as the main baseline. As an additional experiment, we compare it with a global-advantage variant

$$-A_t \log \lambda_{k_t}, \qquad A_t = r_t - b_t,$$

where b_t is a global exponential moving average of reward. The baseline depends only on the global reward history, so the change is the standard variance-reduction form with the same credit assignment rule.

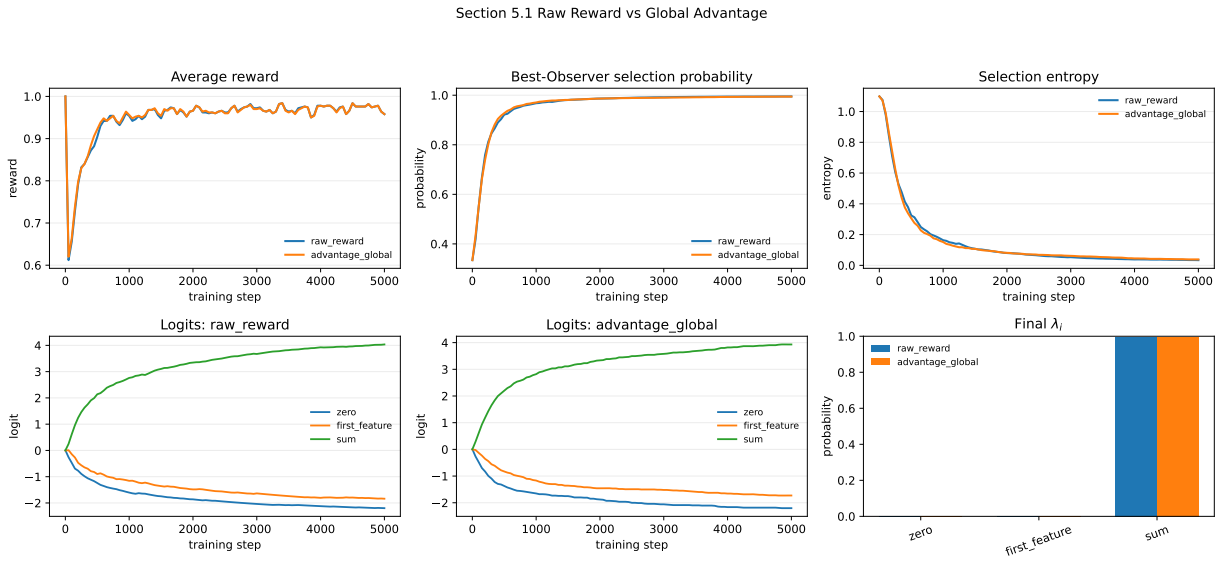


Figure 21: Raw reward versus global advantage in the positive-reward setup. Both updates concentrate on the known best Observer. The advantage update changes the logit path and ends with almost the same end-of-run selection probability.

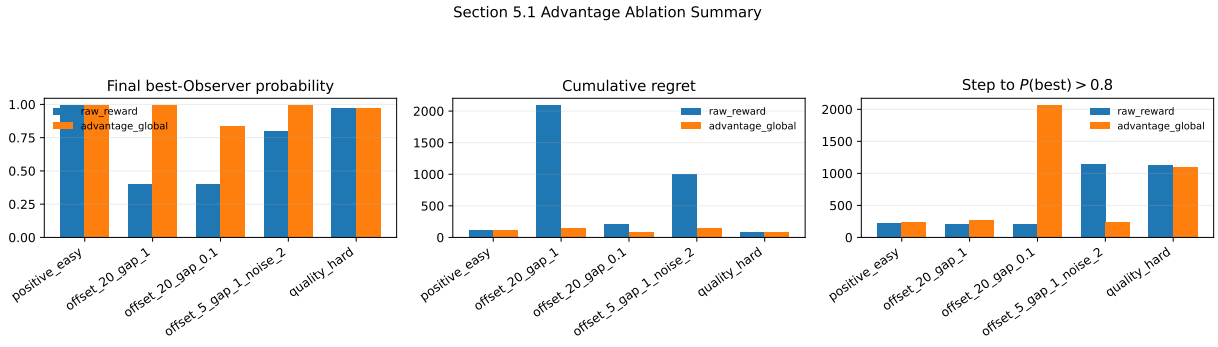


Figure 22: Summary of the raw-reward and advantage comparison across selected conditions. The two updates are close in the original low-offset task, but the advantage update is more stable when rewards contain a large common positive offset.

Condition	Update	Final reward	Final P(best)	Cumulative regret	Step to P(best) > 0.8	P(best) std
positive, gap 1	raw reward	0.958	0.995	116.1	231.0	0.001
positive, gap 1	advantage	0.958	0.994	110.3	245.0	0.000
offset 5, gap 1	raw reward	5.958	0.996	655.5	1149.4	0.002
offset 5, gap 1	advantage	5.958	0.994	117.9	238.0	0.000
offset 20, gap 1	raw reward	20.580	0.400	2097.5	208.8	0.490
offset 20, gap 1	advantage	20.958	0.995	151.1	276.6	0.001
offset 5, gap 0.1	raw reward	5.068	0.598	186.7	1394.2	0.486
offset 5, gap 0.1	advantage	5.091	0.925	76.9	2298.0	0.011
offset 5, gap 1, noise 2	raw reward	5.796	0.799	1005.7	1143.8	0.399
offset 5, gap 1, noise 2	advantage	5.940	0.996	144.7	241.6	0.002
quality hard	raw reward	0.474	0.972	86.8	1125.4	0.005
quality hard	advantage	0.474	0.970	83.2	1105.0	0.006

Table 20: Raw-reward versus global-advantage comparison over 5 seeds. The offset conditions add the same constant to all rewards, so the best Observer is unchanged.

In the original low-offset selection task, the raw-reward and advantage updates are nearly identical. The reward scale is small, the best Observer is well separated, and the raw estimator has little variance to remove. The difference appears when the reward table contains a large common positive offset, such as rewards near 20 versus 21. That offset leaves the best Observer unchanged and contributes little useful gradient in expectation, but it multiplies the sampled log-probability and makes the raw update noisy. The global advantage subtracts most of this common component. In the offset-20, gap-1 test, raw reward ends with $P(\text{best}) = 0.400$ and high seed variance, while the advantage update ends with $P(\text{best}) = 0.995$. The step-to-threshold column can be misleading in these offset runs because the raw update can briefly cross the threshold and later move to a different Observer. The end-of-run best-Observer probability, seed variance, and cumulative regret give the more useful comparison.

The paper update should remain the raw-reward form because it is the simpler baseline and matches the stated raw-reward rule. The advantage form is useful as an appendix variance-reduction variant for reward designs with large common offsets, small quality gaps, or high reward noise.

Imperfect-Reward Joint Scaling Experiments

This subsection folds in the imperfect-reward scaling experiments. The setting is still the reward-following track. One Observer watches many Actors. Two Actors are genuinely good: they see the state and can learn a state-dependent policy. The other Actors are blind distractors: their input is zeroed, so they can only learn one constant action. The Observer has following budget B, assigns following rates μ_j, and reports reward to Actor j with probability

$$q(\mu_j) = 1 - e^{-c\mu_j}.$$

A good Actor only takes a policy-gradient step when this report happens.

The earlier one-axis experiments were mostly stable. Scaling states alone worked. Scaling the actor crowd alone mostly worked. The hard case was scaling states and Actors together at a fixed budget. On a ladder from 16 states and 8 Actors to 144 states and 128 Actors, good-Actor accuracy and budget concentration both dropped sharply.

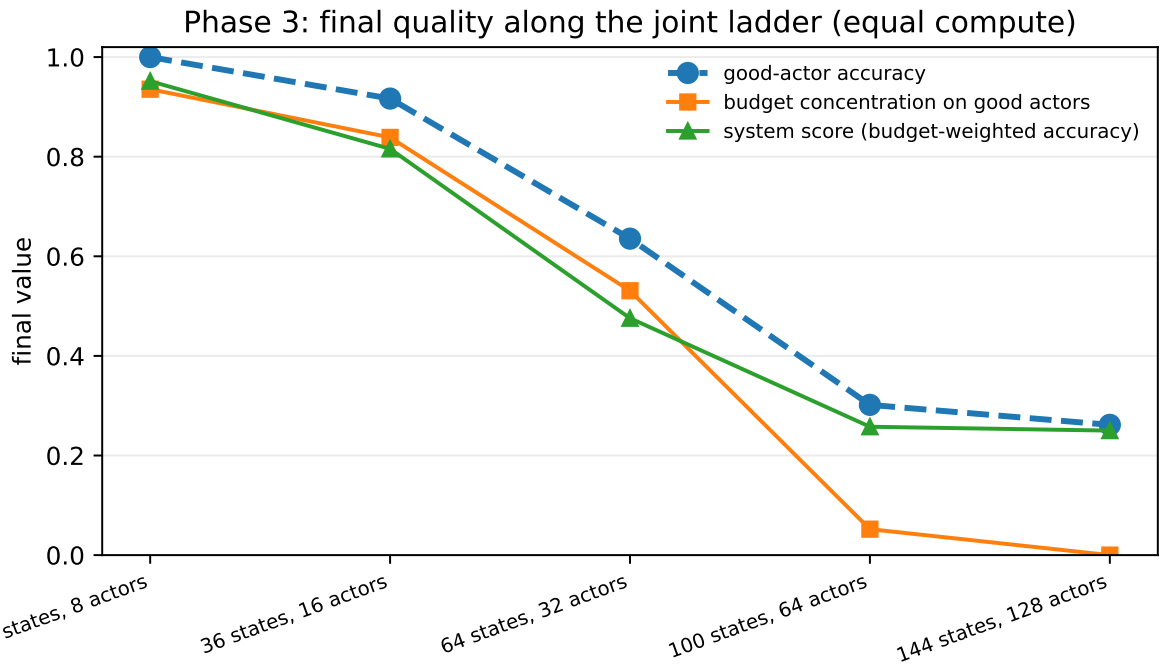


Figure 23: Joint state-and-Actor scaling at fixed budget $B = 2$. Good-Actor accuracy, budget concentration, and the concentration-weighted system score all drop sharply past the 64-state and 32-Actor rung.

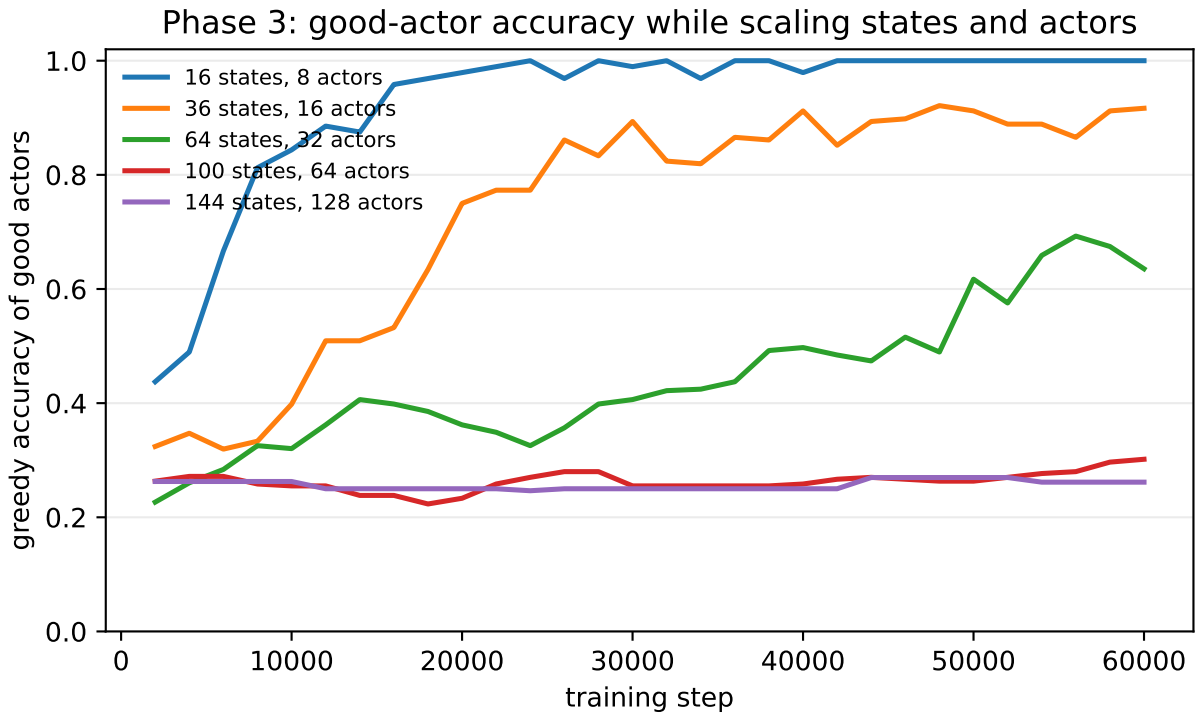


Figure 24: Joint state-and-Actor scaling trajectories. The large-rung curves show the cliff throughout training and at the end of the run.

States	Actors	Good-Actor accuracy	Budget concentration	System score
16	8	1.000	0.935	0.952
36	16	0.917	0.839	0.816
64	32	0.635	0.531	0.476
100	64	0.302	0.052	0.258
144	128	0.262	0.000	0.250

Table 21: Joint imperfect-reward scaling at fixed $B = 2$, 60000 steps, and 3 seeds. Good-Actor accuracy is the actual policy-learning metric. Budget concentration is the fraction of following budget on the two good Actors.

Perfect-reward sanity experiment. The first experiment hard-codes the report probability to one. This means every acting Actor always receives its reward feedback. It is the $B \to \infty$ limit of the reporting rule, so it gives a ceiling for what the imperfect-reward mechanism should approach when reporting is frequent enough. With perfect reward, the joint cliff mostly disappears: accuracy stays high where the fixed-budget run falls near chance.

states	actors	perfect good-acc	$B = 2$ good-acc
16	8	1.000	1.000
36	16	0.986	0.907
64	32	0.785	0.638
100	64	0.792	0.302
144	128	0.707	0.262

Table 22: Perfect reward versus $B = 2$ along the joint ladder. Perfect reward removes the sharp cliff, showing that the update rule has enough signal when reward reports are frequent.

Budget experiment. At the collapsed 100-state and 64-Actor rung, increasing B raises the report probability on good Actors and raises good-Actor accuracy toward the perfect-reward ceiling. This also explains the earlier confusion about concentration: concentration can fall when B is larger because the budget is spread over more Actors, but good-Actor accuracy still improves.

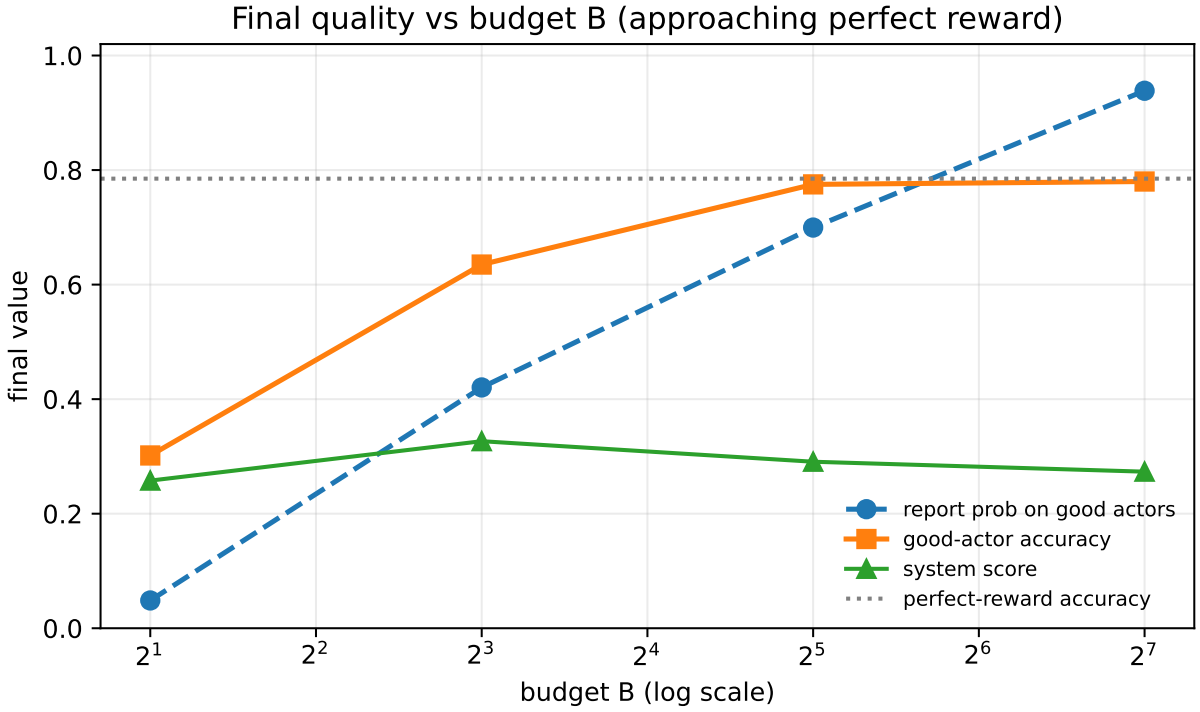


Figure 25: Budget experiment at 100 states and 64 Actors. As B grows, report probability on the good Actors and good-Actor accuracy rise toward the perfect-reward line.



Figure 26: Budget experiment trajectories at 100 states and 64 Actors. Higher budget gives the good Actors more reward reports during training, and their accuracy improves with that extra feedback.

condition	report prob. good	good-acc	concentration	system score
perfect ($\theta = 1$)	1.000	0.803	0.673	0.627
$B = 2$	0.048	0.302	0.052	0.258
$B = 8$	0.420	0.635	0.152	0.327
$B = 32$	0.703	0.788	0.077	0.292
$B = 128$	0.940	0.793	0.044	0.274

Table 23: Budget experiment at 100 states and 64 Actors. "Report prob. good" is the mean reward-report probability for the two good Actors.

Learning-rate experiment. Peter's main instruction was to look at the learning-rate curve before calling a case broken. Bo therefore swept the Actor learning rate under perfect reward at the same 100-state and 64-Actor rung. The curve has the expected shape: too small crawls, the middle range works best, and too large locks into a bad policy.

Figure file `learning_rate_experiment_trajectories` is not in the repository yet.

Figure 27: Good-Actor accuracy over training for each Actor learning rate under perfect reward. The middle rates climb; the largest rate moves badly and settles near chance.

learning rate	good-acc	late wobble
0.001	0.623	0.028
0.003	0.813	0.017
0.010	0.803	0.035
0.030	0.748	0.046
0.100	0.250	0.000

Table 24: Actor learning-rate experiment under perfect reward at 100 states and 64 Actors. Late wobble is the standard deviation of accuracy over late snapshots.

Actor-selection dilution. Even under perfect reward, uniform Actor sampling gives each good Actor only a small share of the active updates in a fixed-length run. With 64 Actors, the two good Actors together receive only 2 out of 64 turns under uniform sampling. Giving the good Actors more turn probability closes most of the residual gap.

Figure file `actor_selection_experiment` is not in the repository yet.

Figure 28: Good-Actor accuracy versus the selection mass placed on the good Actors. The dotted line is the uniform 2 out of 64 baseline. More turns per good Actor recover near-perfect accuracy.

good-Actor selection share	turns per good Actor	good-acc
0.031 (uniform 2 out of 64)	938	0.813
0.10	3000	0.968
0.25	7500	0.977
0.50	15000	0.962

Table 25: Perfect reward, 100 states and 64 Actors, learning rate 0.003. The residual gap is mostly the number of turns each good Actor gets.

Putting it together. With learning rate 0.003, budget $B = 32$, and enough total steps, the once-collapsed rung works again under the ordinary imperfect-reporting rule.

steps	report prob. good	good-acc	system score
60,000	0.629	0.732	0.280
300,000	0.743	0.930	0.308

Table 26: Imperfect reward, 100 states and 64 Actors, learning rate 0.003, $B = 32$. With enough compute the previously collapsed rung reaches 0.93 accuracy.

The conclusion is narrower than the first scaling read. The cliff came from three controllable inputs: learning rate, reporting budget, and the number of turns each good Actor receives. With those set correctly, the imperfect-reward side recovers.

Systematic per-axis stress test with scalar state ID and embedding. As an additional experiment under the representation used for the imperfect-reward runs, Bo scaled one axis at a time. The state input is an integer ID $0, \ldots, K-1$. The policy first maps that ID to a learnable embedding vector, then applies a linear policy head. Uniform action exploration anneals from $\varepsilon = 0.1$ to 0 over training. A row is marked `pass` when the settled optimal-action probability p_{opt} is near its ceiling, `watch` when it is lower but still usable, and `rerun` when it needs more tuning.

States. This experiment uses one Actor, perfect reward, and K states with K actions. It isolates the policy head on scalar IDs. A plain embedding reaches $p_{\text{opt}} = 1.0$ through $K = 16$. At $K = 32$ it drops to 0.833, which is a `watch` row. The same embedding fits the diagnostic lookup through much larger K. This points to sampled policy-gradient variance as the limiting factor. An MLP on the raw scalar learns less reliably as K grows.

policy head	$K = 3$	$K = 8$	$K = 16$	$K = 32$
scalar plus embed	1.000	1.000	1.000	0.833
scalar plus embed plus MLP1	1.000	1.000	0.750	0.271
scalar plus rbf	1.000	1.000	0.937	0.687
scalar plus MLP1, normalized	0.889	0.458	0.187	0.052
scalar plus MLP2, normalized	0.778	0.167	0.083	0.031

Table 27: States axis under scalar IDs: mean p_{opt} over 3 seeds. The plain embedding is the strongest policy head in this experiment.

Actors. This experiment uses one Observer, N Actors, and fixed $K = 3$. The learned objects are the following rates μ and the Actor policies, as in the one-Observer imperfect-reward joint case. Mean and minimum p_{opt} stay above 0.996 through $N = 32$.

actors N	mean p_{opt}	min p_{opt}	verdict
4	1.000	1.000	pass
8	1.000	0.999	pass
16	0.999	0.999	pass
32	0.998	0.996	pass

Table 28: Actor-axis stress test: one Observer and N Actors, scalar-ID embedding policy, $K = 3$, 40000 steps, 3 seeds.

Observers. This experiment fixes 8 Actors and $K = 3$, then varies the number of Observers M. All runs keep $p_{\text{opt}} = 1.0$ through $M = 32$.

observers M	mean p_{opt}	min p_{opt}	verdict
4	1.000	1.000	pass
8	1.000	1.000	pass
16	1.000	1.000	pass
32	1.000	1.000	pass

Table 29: Observer-axis stress test: 8 Actors and M Observers, scalar-ID embedding policy, $K = 3$, 40000 steps, 3 seeds.

Taken together, under scalar state IDs with an embedding policy head, the imperfect-reward system scales well when only states, only Actors, or only Observers change. The only `watch` row is the large-K states-axis policy run, and the earlier joint rung recovers once learning rate, budget, and per-Actor compute are set correctly.

Imperfect-State Records

Each imperfect-state run stores a CSV trajectory, a JSON artifact, and a checkpoint. The artifact records the run configuration, derived stream seeds, initial and final compression weights, emitted messages for every state, adaptive-policy centers and widths, action-layer parameters,

source-selection probabilities, and optimizer state. The following quantities are used in the main tables.

- **Lowest correct-action probability** is the smallest probability assigned to the target action over all relevant states and Observers.
- **Regions below** 0.80 counts state–Observer pairs whose target-action probability is below 0.80.
- $P(\text{full})$ is the probability of selecting the full-state Actor; group rows average this quantity over Observers.
- **Message movement** is the root-mean-square change in the full source's messages between adjacent saved checkpoints.
- **Relative parameter movement** divides a parameter change by the current parameter norm, allowing compression, policy, and selection parameters with different dimensions to use the same final-window threshold.

The report uses a correctness threshold of 0.80 for a state-action region. This threshold is applied to the learned policy without the exploration mixture. The exploration floors remain active only while sampling actions and message sources during training.

The one-to-one diagonal scale ladder and ceiling rulers are summarized from `outputs/scale_ladder_20260811/`. The five-seed $K = 64, A = 16$ compression-and-policy rerun is stored under `outputs/rng_clean_k64_a16_phi_the`